RAPPORT DE STAGE DE FIN D'ÉTUDES

Présenté en vue de l'obtention du

DIPLÔME D'INGÉNIEUR D'ÉTAT

Filière : Systèmes d'Information et Transformation Digitale

Développement d'un système intelligent de surveillance et d'alerte de la qualité de l'air intérieur

Réalisé par :

ISSA MOEVA Aïssa

Encadré par :

Pr. *Karim EL BOUCHTI*

Dr Ahmed SLALMI

Soutenu le 07/07/2026 devant le Jury composé de :

Pr. Karim EL BOUCHTI	Professeur, Ecole Supérieure de Technologie de salé.
Pr. Abderrahmane EZZAHOUT	Professeur, Faculté des Sciences de Rabat.

Année Universitaire : 2025 – 2026

Dédicaces

Je dédie ce travail à mes parents **ISSA MOEVA** et **NAFISSA MZE** pour leur soutien indéfectible depuis le début de mon parcours, qui m'ont supporté et encouragé et qui ne cessent de le faire depuis les Comores**.**

Je le dédie également à mes frères et sœurs qui m'ont porté leur soutien moral et financières, qui ont su me conseiller à chaque moment de besoin. Je le partage également à mes sœurs pour leur encouragement.

Je le dédie également à mes encadreurs qui mon accompagné dans ce travail.

Je le dédie à mes professeurs de l'UPF au campus de Rabat, à mes camarades qu'on a partagé la même classe et nos savoirs.

Je dédie ce mémoire à toute personne qui consultera ce document en espérant lui offrir de l'intérêt ainsi qu'à tous ceux qui suivent cette voix.

Remerciements

Je remercie ALLAH le Clément et Miséricordieux en reconnaissance de cette vie et de m'avoir permis à réaliser ce parcours.

Je remercie mes parents et mes proches pour leur soutien constant sans faille, pour leur encouragement, leur engagement et leurs conseils qui m'ont été favorable.

Je porte aussi ma reconnaissance envers mon maitre de stage le **Dr Ahmed SLALMI** et son entreprise qui m'ont ouvert les portes et m'ont offert leur savoir précieux, je le porte également à mon encadreur pédagogique le **Professeur Karim ELBOUCHTI** qui m'a suivi tout au long de la rédaction et de la finalisation de ce mémoire.

Mes remerciements aux membres du jury composé du **Pr Karim ELBOUCHTI** et du **Pr Abderrahmane EZZAHOUT** pour l'intérêt, remarques et suggestions qu'ils porteront sur ce travail.

Mes remerciements à l'administration de l'Université Privée de Fès au camps de Rabat et de Fès pour ce programme enrichissant et plein de savoir moderne et précieux, à tout le personnel du campus de Rabat qui nous a toujours soutenu et conseillé tout au long de notre parcours, mention spéciale à notre directrice **Mme Mouna CHAKIR**.

Je remercie également **Dr. ALIHAMIDI,** compatriote pour ses précieux conseils pendant l'accomplissement de ce projet de fin d'études.

Mes remerciements à toute ma communauté et toute personnes de près ou de loin qui m'a aidé pendant mon parcours.

Résumé

Ce projet de fin d'étude, réalisé dans le cadre de la filière **Systèmes d'Information et Transformation Digitale,** a pour objectif la conception et la mise en œuvre d'un prototype de **système intelligent de surveillance de la qualité de l'air intérieur.**

Le dispositif repose sur une carte ESP32 et une batterie de capteurs (MQ2, MQ7, MQ135 et DHT22) permettant de mesurer en temps réel la présence de gaz nocifs et inflammables, tout en corrigeant les valeurs influencées par la température et l'humidité. Les alertes sont déclenchées localement par un LED et un buzzer, et transmises à distance via Telegram, tandis que les données sont stockées dans une base en ligne pour un suivi continu sur une plateforme web.

Les tests de validation ont confirmé la pertinence du prototype, qui se distingue par sa précision, son faible coût et son ouverture technologique, répondant ainsi aux enjeux sanitaires et sécuritaires liés à la pollution de l'air intérieur.

Mots clés : Qualité de l'air intérieur (QAI), Internet des objets (IoT), Système de surveillance, ESP32, capteurs de gaz.

Abstract

This final year engineering project, conducted within the **Information Systems and Digital Transformation** program, aims to design and implement a prototype of an **IoT based indoor air quality monitoring system**.

The solution is built around the ESP32 microcontroller and set of sensors (MQ2, MQ7, MQ135, DHT22), enabling real time detection of harmful and flammable gases. Sensor readings are corrected for temperature and humidity variations, while alerts are triggered both locally by LED and Buzzer, and remotely via Telegram API. Data is stored in an online database and visualized through a dashboard for continuos monitoring in a web application.

Validation tests demonstrated the efficiency of the prototype, which combines accuracy, affordability, and openness, highlighting the potential of IoT technologies to address public health and safety challenges in indoor environments.

Keywords : Indoor Air Quality (IAQ), Internet of Things (IoT), ESP32, Gas sensors.

Tables des matières

DEDICACES I

REMERCIEMENTS II

RESUME III

ABSTRACT IV

TABLES DES MATIERES V

LISTES DES FIGURES VII

LISTE DES TABLEAUX VIII

LISTE DES ABREVIATIONS IX

INTRODUCTION GENERALE 1

CHAPITRE 1 : CONTEXTE ET ETAT DE L'ART 3

1.1 CONTEXTE DU PROJET 4
1.2 ETAT DE L'ART 5
1.2.1 Pourquoi surveiller l'air à l'intérieur 5
1.2.2 Choix des équipements 5
1.2.3 Le problème de la température et de l'humidité 8
1.2.4 Comment envoyer des alertes à distance 9
1.2.5 Le stockage 10
1.2.6 Notre valeur ajoutée 10
1.3 LA SOCIETE D'ACCUEIL 10
CONCLUSION DU PREMIER CHAPITRE 11

CHAPITRE 2 : CONCEPTION ET REALISATION DU PROTOTYPE 12

INTRODUCTION 13
2.1 ARCHITECTURE ET DIAGRAMMES DU SYSTEME 13
2.1.1 Architecture générale du système 13
2.1.2 Diagramme de fonctionnement du système 14
2.1.3 Diagramme de classe 15
2.1.4 Diagramme de cas d'utilisation 16
2.1.5 Diagramme de séquence 17
2.2 REALISATION DU PROTOTYPE 18
2.2.1 Conception matérielle 18
2.2.2 Intégration embarquée 21
2.2.3 Ecosystème logiciel 29
CONCLUSION DU CHAPITRE 2 34

CHAPITRE 3 : RESULTATS ET TEST DE VALIDATION DU PROTOTYPE 35

INTRODUCTION 36
3.1 SCENARIO DE TEST 36

3.2 RESULTAT OBTENU 37
3.3 LIMITES ET AMELIORATIONS 41
CONCLUSION DU CHAPITRE 3 42
GESTION DE PROJET 43
CONCLUSION GENERALE 44
REFERENCES 45
ANNEXES 47
ANNEXE A : LES CAPTEURS 47
A.1 — CAPTEUR DE GAZ MQ-2 47
A.2 — CAPTEUR DE MONOXYDE DE CARBONE MQ-7 48
A.3 — CAPTEUR DE QUALITE DE L'AIR MQ-135 49
A.4 — CAPTEUR DE TEMPERATURE ET D'HUMIDITE DHT22 51
A.5 — TABLEAU COMPARATIF DES CAPTEURS 52
ANNEXE B : LE MICROCONTROLEUR 53
B.1 — CARTE ESP32 53
B.2 — JUSTIFICATION DU CHOIX 54
ANNEXE C : LE SYSTEME D'ALERTE LOCAL 56
C.1 — LE BUZZER 56
C.2 — LA LED DE SIGNALISATION 57
ANNEXE D : CORRECTION DES MESURES (TEMPERATURE ET HUMIDITE) 58
D.1 — METHODES DE CORRECTION ETUDIEES 58
D.2 — PRINCIPE DE LA CORRECTION LINEAIRE 58
ANNEXE E : SCHEMA DE CABLAGE ET BROCHAGE 60
ANNEXE F : LA COMMUNICATION (PROTOCOLE MQTT) 61
F.1 — LE PROTOCOLE MQTT 61
F.2 — ARCHITECTURE PUBLISH/SUBSCRIBE 61
F.3 — NIVEAUX DE QUALITE DE SERVICE (QOS) 62
F.4 — CARACTERISTIQUES TECHNIQUES DU PROTOCOLE 62
F.5 — CONFIGURATION DU BROKER DU PROJET 62
ANNEXE G : PLATEFORME AIRWATCH ET ALERTES DISTANTES 64
G.1 — LA PLATEFORME WEB AIRWATCH 64
G.2 — LES ALERTES DISTANTES PAR TELEGRAM 65
ANNEXE H : ÉCOSYSTEME LOGICIEL ET OUTILS 66
H.1 — FRAMEWORKS DE LA PLATEFORME AIRWATCH 67
ANNEXE I : COMPARATIF AVEC LES SYSTEMES EXISTANTS 68
ANNEXE J : JEU DE VALEURS DE TEST (SIMULATION) 69
ANNEXE K : GLOSSAIRE DES ABREVIATIONS 70

Listes des figures

Figure 1: carte ESP32 6
Figure 2: capteurs MQ-2, MQ-7 et MQ-135 6
Figure 3 : capteur DHT22 7
Figure 4 : Buzzer 7
Figure 5 : Architecture générale du système 14
Figure 6: Diagramme de fonctionnement 14
Figure 7 : Diagramme de classe 15
Figure 8: Diagramme de cas d'utilisation 16
Figure 9 : Diagramme de séquence 17
Figure 10: Schéma de branchement des capteurs MQ 19
Figure 11 : Schéma de branchement du capteur DHT22 19
Figure 12 : Schéma de branchement du système d'alerte locale 20
Figure 13: Schéma de branchement du système de la QAI 21
Figure 14 : Console powershell dans docker / connexion à la base de données MySQL 24
Figure 15: console mysql 24
Figure 16: Console mysql (description de la table airquality) 24
Figure 17: Interface du tableau de bord AirWatch 25
Figure 18: Interface du tableau de bord AirWatch 2 25
Figure 19: Interface du tableau de bord AirWatch partie Espaces surveillés 26
Figure 20 : Interface du tableau de bord AirWatch partie Historique 26
Figure 21 : Interface de création du bot Telegram (Botfather) 27
Figure 22 : Interface de création du bot Telegram (Botfather) finalisation 28
Figure 23 : Interface de discussion du bot Telegram 28
Figure 24 : Définition de la fonction d'envoi du message Télégram 29
Figure 25: Définition et contrôle des seuils 29
Figure 26: Logo Canva 29
Figure 27 : Espace de travail de Fritzing 30
Figure 28: Logo Visual Studio Code 30
Figure 29 : Logo micropython 31
Figure 30: Interface de travail de la plateforme draw.io 32
Figure 31 : Espace de travail de Wokwi 32
Figure 32 : Espace de travail de Node-RED 33
Figure 33 : Interface de Docker 33
Figure 34 : Interface de Wokwi pendant un premier test 37
Figure 35 : Interface de Wokwi pendant un second test 37
Figure 36 : Interface de Wokwi pendant un troisième test 38
Figure 37 : Interface de Wokwi pendant un quatrième test 39
Figure 38: Interface de Wokwi pendant un cinquième test / réception d'une notification Télégram 39
Figure 39 : Interface du Bot Télégram réception des données dépassant le seuil 40
Figure 40 : Interprétation des données dans le tableau de bord 40
Figure 41 : Diagramme de gestion de projet 43

Liste des tableaux

Tableau 1: Comparaison de notre projet et des projets existants 9
Tableau 2: Tableau de branchement des composants 20
Tableau 3 : configuration du Wi-Fi 22
Tableau 4 : configuration du protocole de communication MQTT 23
Tableau 5 : Tableau caractéristique du capteur MQ-2 48
Tableau 6 : Tableau caractéristique du capteur MQ-7 49
Tableau 7 : Tableau caractéristique du capteur MQ-135 50
Tableau 8 : Tableau caractéristique du capteur de température et d'humidité DHT22 51
Tableau 9 : Tableau comparatif des capteurs 52
Tableau 10 : Tableau caractéristique de la carte ESP32 54
Tableau 11 : Tableau caractéristique du Buzzer 56
Tableau 12 : Tableau caractéristique de la LED 57
Tableau 13 : Tableau significatif de la formule de correction 59
Tableau 14 : Tableau de cablage 60
Tableau 15 : Tableau de la qualité of Service 62
Tableau 16 : Tableau caractéristique du protocole MQTT 62
Tableau 17 : Tableau caractéristique du Broker MQTT 63
Tableau 18 : Tableau caractéristique du système d'alerte distante 65
Tableau 19 : Tableau caractéristique de l'écosystème logiciel 66
Tableau 20 : Tableau caractéristique des technologies utilisées 67
Tableau 21 : Tableau comparatif avec les systèmes existant 68
Tableau 22 : Tableau de valeur des tests de validation 69
Tableau 23 : Tableau des abréviations 70

Liste des abréviations

API : Application Programming interface

AVC : Accident Vasculaire Cérébrale

CO_2 / CO : Dioxyde de Carbone / Monoxyde de Carbone

COV : Composé Organique Volatile

GPIO : General Purpose Input/Output

IDE : Interated Development Environment

IoT : Internet of Things

LIE/LSE : Limite Inférieur/Supérieur d'Explosivité

MQTT : Message Queuing Telemetry Transport

PFE : Projet de Fin d'Etude

ppm : partie par millions

QAI : Qualité de l'Air Intérieur

SSID : Service Set Identifier

UPF : Université Privée de Fès

Introduction générale

La qualité de l'air intérieur (QAI) s'est imposée au cours des dernières décennies comme un enjeu critique, dépassant le simple cadre du confort thermique pour devenir une question majeure de santé publique, de performance économique et de sécurité environnementale. Les données statistiques récentes, notamment celles de l'Organisation Mondiale de la Santé[1], soulignent une réalité alarmante : chaque année, des milliers de personnes sont intoxiquées par le monoxyde de carbone ou victimes d'explosions dues à des gaz inflammables dans des espaces clos (domiciles, établissements scolaires, lieux de travail, ateliers, cuisines). Ces accidents, souvent tragiques et parfois mortels, pourraient dans une large mesure être évités grâce à des systèmes de surveillance préventifs, fiables, accessibles et capables d'anticiper les risques plutôt que de simplement les constater[2].

Au-delà des impacts sanitaires directs, la QAI et les paramètres météorologiques associés (température, humidité relative) jouent un rôle déterminant sur la productivité, la concentration et le bien-être des occupants. Une atmosphère dégradée peut altérer les capacités cognitives, réduire l'efficacité du travail et même affecter le développement cognitif et les performances scolaires des enfants[3]. Face à ces constats, la surveillance de la QAI devient une nécessité impérieuse, de plus en plus encadrée par la réglementation (exemple : Décret n° 2021-1105 en France)[4]. Cependant, les dispositifs actuels sur le marché sont souvent onéreux, propriétaires et souffrent de lacunes techniques. Par exemple, l’absence de correction des données selon les conditions ambiantes conduit souvent à des lectures erronées, un problème bien documenté pour les capteurs à oxyde métallique[5].

C'est dans ce contexte technique et sociétal que s'inscrit notre **Projet de Fin d'Études (PFE)**, réalisé dans le cadre de notre dernière année du cycle ingénieur à l'**Université Privée de Fès (UPF)**, campus de Rabat, au sein de la filière **Systèmes d'Information et Transformation Digitale**. Ce projet a pour ambition de concevoir et de réaliser un prototype de système de surveillance de la qualité de l'air dans un espace clos, capable de pallier les déficiences des solutions existantes en combinant précision, autonomie, connectivité et accessibilité économique et un suivi en temps réel.

L'objectif principal de ce travail est de développer une solution intégrée, à la fois précise, économique et ouverte, basée sur des technologies libres et accessibles. Le système repose sur une architecture électronique intelligente centrée autour du microcontrôleur **ESP32**, capable de

surveiller en continu la qualité de l'air et de détecter une gamme étendue de polluants grâce à une batterie de capteurs spécialisés avec une plateforme de suivi dédiée et personnalisée.

Notre démarche méthodologique s'articule autour de trois grands chapitres. Le premier chapitre dresse l'état de l'art sur les technologies de capteurs et les normes de QAI, analyse les solutions concurrentes et présente la structure d'accueil de notre stage. Le deuxième chapitre est ensuite consacré à la conception et à la réalisation technique du prototype, en détaillant à la fois l'architecture matérielle (capteurs MQ, DHT22) et l'architecture logicielle (firmware, intégration de l'API Telegram pour les alertes et la plateforme dédiée). Enfin, le troisième chapitre expose les résultats des tests expérimentaux menés pour valider la précision des mesures corrigées et la fiabilité du système, avant de conclure sur les limites de notre approche et ses perspectives d'industrialisation. À travers ce travail, nous espérons démontrer la faisabilité d'une solution accessible et illustrer l'apport concret de l'Internet des Objets (IoT) dans la protection de la santé publique.

Chapitre 1 : Contexte et Etat de l'art

Avec l'enjeu du réchauffement climatique, être conscient de l'air dont on est exposé devient crucial pour tous. Il nous permettra de limiter nos ressources polluantes qui auront un impact sur notre santé et bien-être.

1.1 Contexte du projet

Depuis les débuts du XIXème siècle, une forte croissance de la pollution de l'air fut enregistrée. C'est en 1958 que l'organisation mondiale de la santé a publié le premier rapport technique traitant de la pollution de l'air et son effet sur la santé marquant ainsi une prise de conscience internationale[6]. Aujourd'hui, la pollution de l'air est reconnue comme étant l'un des principaux risques sanitaires, avec une charge de morbidité comparable à celles du déséquilibre de l'alimentation et du tabagisme ; elle est, avec le changement climatique la plus importante menace environnementale à l'origine de plus de 7 millions de décès par an[1].

En 2019, d'après les estimations de l'OMS, 4,2 millions de décès prématurés étaient attribuables à la pollution de l'air ambiant (extérieur) et 3,2 millions de décès à la pollution de l'air intérieur domestique[1]. Ces décès sont liés à des maladies cardiovasculaires (AVC, cardiopathie ischémique) et respiratoires (infection respiratoire) ainsi qu'à des cancers.

En outre la pollution de l'air a également été reliée au développement ou à l'aggravation de maladies telles que l'asthme ou le diabète, le cancer des poumons et des voies urinaires/vessie d'après le CIRC (centre international de la recherche sur le cancer)[7] et de 86 millions de perte d'années de vie en bonne santé à l'échelle mondiale [1].

Malgré l'ampleur de ces enjeux, l'accessibilité de solution de monitoring de la qualité de l'air pour chaque citoyen de différente classe social reste un défi qui suscite d'être relevé. Les systèmes professionnels sont souvent coûteux et centralisés, créant une inégalité d'accès à l'information sanitaire critique.

C'est dans ce contexte que s'inscrit notre projet dont l'objectif est de tenir et mettre les citoyens hors du danger de la pollution de l'air dans les espaces clauses en concevant un système de monitoring intelligent qui effectuera des analyses en temps réel de la qualité de l'air. Dans ce projet nous allons concevoir un prototype fonctionnel d'un dispositif de détection en temps réel des gaz nocifs et inflammables, déclencher des alarmes locales selon les seuils définis, envoyer des alertes à distance (notification Telegram), stocker les mesures pour analyse et suivi et fournir un prototype fonctionnel et documenté.

1.2 Etat de l'art

Dans notre projet de fin d'études, nous devons concevoir un système capable de surveiller la qualité de l'air dans une salle fermée. L'objectif est de détecter des gaz dangereux comme le monoxyde de carbone, les gaz inflammables ou les composés organiques volatiles. Si les seuils sont dépassés, le système doit déclencher une alarme locale et envoyer une alerte à distance via Wi-Fi.

Avant de commencer à fabriquer notre prototype, nous avons d'abord cherché ce qui existe déjà. Ça nous a permis de comprendre les capteurs disponibles, les systèmes déjà réalisés, et les limites des solutions actuelles.

1.2.1 Pourquoi surveiller l'air à l'intérieur

On pense souvent que l'air intérieur est plus propre que l'air extérieur. Mais cette idée n'est pas totalement vraie.

Selon l'Organisation Mondiale de la Santé (OMS), une mauvaise qualité de l'air intérieur peut provoquer des maladies respiratoires, des maux de tête, des allergies, et même des intoxications graves [1].

Dans les pièces fermées, plusieurs gaz peuvent devenir dangereux :

- Le monoxyde de carbone : il est inodore et peut tuer en quelques heures
- Les gaz inflammables : ils peuvent exploser s'ils s'accumulent
- Les composés organiques volatils : ils irritent les yeux et les voies respiratoires

C'est pour cette raison que notre projet a du sens : il peut protéger les personnes dans une salle de classe, un bureau, une cuisine ou un atelier.

1.2.2 Choix des équipements

1.2.2.1 Une carte ESP32

C'est une carte de développement basée sur le système sur une puce ESP32, qui est un microcontrôleur à double cœur, doté de fonctionnalité Wi-Fi et Bluetooth intégrées. Elle est largement utilisée dans le développement de projets IoT grâce à ses capacités de connectivité sans fil, sa faible consommation d'énergie et sa flexibilité. Elle dispose de nombreux ports d'entrée/sortie pour connecter différents capteurs, actionneurs et périphériques, tels que les capteurs de température, des afficheurs LED, …. Elle est compatible à de nombreuses

plateformes de développements logiciel telles que Arduino, Micropython, ESP-IDF et autres[8].



Figure 1: carte ESP32

1.2.2.2 Les capteurs sensoriels

Dans ce projet, nous avons une liste de capteurs précis que nous devons raccorder à notre microcontrôleur pour capturer des données spécifiques.

Sur cette liste nous avons :

- Des capteurs de la famille MQ pour la détection des gaz nocifs
- Un capteur de température et d'humidité

1.2.2.2.1 Les capteurs MQ

Dans cette famille de capteurs, nous allons utiliser principalement les capteurs MQ-2, MQ-7, MQ-135[9].



Figure 2: capteurs MQ-2, MQ-7 et MQ-135

1.2.2.2.2 Le capteur de température et d'humidité

Nous avons choisi le capteur DHT22 pour sa performance et sa précision par rapport à d'autres capteurs de mesure de température ambiante.

DHT22 est un capteur de température et d'humidité avec des caractéristiques très proches de la haute précision. Il utilise un capteur capacitif pour mesurer l'humidité et un thermistor pour mesurer la température, fournissant des lectures numériques précises via un signal numérique[10].



Figure 3 : capteur DHT22

1.2.2.3 Système d'alerte local

Pour ce système nous avons opté pour l'utilisation d'un buzzer et un LED de couleur rouge. Un buzzer est un dispositif qui produit un son ou un bruit de bourdonnement. Il est souvent utilisé comme dispositif de signalisation, par exemple dans notre cas pour indiquer si un gaz a été détecté ou a dépassé le seuil négligeable. [11]



Figure 4 : Buzzer

1.2.3 Le problème de la température et de l'humidité

Après la lecture des fiches techniques des capteurs MQ, nous avons compris qu'ils sont très influencés par la température et l'humidité.

Cela signifie que si la pièce est chaude ou humide, le capteur peut donner une mauvaise valeur, même si l'air est propre.

Selon plusieurs études, sans correction, l'erreur de mesure peut atteindre 30 à 50%.

Il faut donc absolument corriger les mesures. Dans la littérature nous avons trouvé les méthodes de correction suivantes :

- La correction linéaire simple
- L'utilisation de tables de correspondance
- Des modèles mathématiques plus complexes

Pour notre projet, nous utiliserons une correction linéaire simple car elle est facile à programmer et suffisante pour un prototype.

La formule est donnée par :

$$\text{ppm_corrigé} = \frac{\text{ppm_brut}}{1 + a(T - 25) + b(HR - 50)}$$

Où T est la température, ppm_brut est la concentration de gaz mesurée directement par le capteur, avant toute correction ou modification par un calcul informatique. C'est la valeur "brute d'usine", qui contient encore des erreurs dues à la température ou à l'humidité, HR est l'humidité, a et b sont les coefficients de correction propres aux capteurs ou au protocole utilisé[12].

Nous avons observé plusieurs systèmes déjà construits par d'autres étudiants ou par des entreprises qui nous a mené au tableau suivant :

Tableau 1: Comparaison de notre projet et des projets existants

Système	Ce qu'il fait	Ce qui manque
Détecteur CO domestique	Alarme locale seulement	Pas d'alerte à distance
Projet AirPi	Mesures + interface web	Assez cher
Système avec ESP8266 et MQ135	Envoi vers le cloud	Pas d'alarme locale
Notre projet (ESP32 + MQ2/MQ7/MQ135 + DHT22)	Alarmes locales et distantes + correction + stockage	Prototype à valider

Ce que nous avons remarqué, c'est qu'aucun système bon marché ne fait pas à la fois :

- Détecter plusieurs gaz
- Corriger les mesures
- Afficher dans un tableau de bord pour le suivi en temps réel
- Alerter localement et à distance
- Stocker l'historique

Nous avons essayé dans notre projet de tout regrouper.

1.2.4 Comment envoyer des alertes à distance

Nous avons étudié plusieurs façons d'envoyer des alertes à distances via le module Wi-Fi de la carte ESP32.

- Notification Telegram :

Telegram permet de créer un bot gratuit. Le bot peut envoyer un message sur notre téléphone quand un seuil est dépassé (Telegram API, 2022).

C'est simple à programmer.

- Plateformes IoT (Blynk, Thingspeak)

Ces plateformes permettent de créer un tableau de bord avec des graphiques. On peut voir les mesures en temps réel depuis un navigateur ou une application, mais certaines fonctionnalités sont payantes.

  - Thingspeak : https://thingspeak.mathworks.com/
  - Blynk : https://www.blynk.io/

- Serveur personnel :

On peut aussi envoyer les données vers un serveurs MySQL et créer notre propre application web ou mobile.

Pour notre projet, nous allons commencer par Telegram car c'est simple et gratuit pour les notifications et AirWatch notre plateforme de suivi en temps réel.

1.2.5 Le stockage

Pour une utilité à l'avenir, nous devons sauvegarder les données de mesures dans un support de stockage en mémoire. 2 possibilités nous font face :

- Stockage sur carte SD : on branche une carte microSD sur la carte ESP32 et on écrit les données dans un fichier texte.
- Base de données en ligne : on envoie les mesures dans une base de données en ligne comme MySQL ou Firebase. Cela permettra de les consulter à distance.

1.2.6 Notre valeur ajoutée

Après étude de l'existant, nous pensons que notre projet peut apporter plusieurs choses :

- La combinaison de plusieurs gaz : monoxyde de carbone, gaz inflammables, composés organiques volatils
- La correction des mesures avec la température et l'humidité
- Les deux types d'alertes : locale et distante
- Un stockage dans une base de données en ligne MySQL et en perspectives dans le cloud
- La visualisation des données en temps réel
- Un coût faible car tous les composants sont moins cher

Aucun des systèmes que nous avons trouvés ne réunit toutes ces caractéristiques à la fois.

1.3 La société d'accueil

Notre projet a été effectué au sein de la Société SLATEL. Cette société intervient dans l'accompagnement technologique et la transformation digitale des organisations. Se positionnant au carrefour de la connectivité et de l'intelligence artificielle, elle déploie une approche globale visant à capter, transporter et valoriser la donnée afin de la transformer en leviers de performance et d'aide à la décision stratégique.

Pour ce faire, l'entreprise apporte une expertise de haut niveau à travers des prestations de conseil et de formation en architectures réseaux informatiques et des télécommunications,

garantissant aux acteurs du secteur une montée en compétences alignée sur les standards internationaux les plus stricts.

En parallèle, elle s'appuie sur une équipe de Data Scientists et d'analystes de données pour mener à bien des projets d'envergure en Data Science, Data Analytics et Business Intelligence. Sa philosophie opérationnelle repose sur un accompagnement de bout en bout, structuré pour garantir la transition fluide d'un besoin théorique vers une solution technologique mature, depuis le cadrage initial jusqu'à la pérennisation par le transfert de compétences. C'est au cœur de cet écosystème pluridisciplinaire, associant la rigueur du conseil en réseaux et la maîtrise technique de l'analyse avancée de données, que s'est déroulé le présent projet.

Conclusion du premier chapitre

Pour conclure, nous avons la conviction que notre projet est utile et original. Les capteurs MQ sont bien adaptés, mais ils ont besoin d'une correction. Les systèmes existants sont souvent incomplets : soit ils n'ont pas d'alerte à distance, soit ils n'ont pas d'alarme locale, soit ils n'enregistrent pas les données.

Notre objectif est donc de construire un prototype complet qui pourra servir de base à un vrai produit de sécurité.

Chapitre 2 : Conception et réalisation du prototype

Introduction

Le projet consiste à concevoir et à réaliser un prototype d'un système électronique intelligent capable de surveiller en continu la qualité de l'air dans une salle fermée (bureau, salle de classe, atelier, cuisine, …).

Le système détecte plusieurs gaz dangereux ou nocifs (CO, gaz inflammables, COV), déclenche des alarmes locales (LED et buzzer) et envoie des alertes à distance via wifi pour prévenir les risques d'intoxication ou d'explosion.

Il dispose également d'une application dédiée (mobile ou web) pour visualiser les mesures en temps réel, consulter l'historique et configurer les seuils d'alerte.

Dans cette partie, nous allons expliquer comment notre système de surveillance et d'alerte de la qualité de l'air est conçu. L'objectif est de décrire l'architecture du système, le choix des composants et le fonctionnement attendu ainsi que les outils utilisés lors de la conception et du développement.

2.1 Architecture et diagrammes du système

2.1.1 Architecture générale du système

Notre système est composé de 3 grandes parties :

- **La partie acquisition :** composé de trois capteurs de gaz (MQ-2, MQ-7 et MQ-135) et d'un capteur d'humidité et de température (DHT22). Ils mesurent la qualité de l'air et envoient des signaux à la carte ESP32.
- **La partie traitement :** composé d'une carte ESP32, qui lit les valeurs des capteurs, calcule les concentrations en partie par millions (ppm), corrige les valeurs avec la température et l'humidité et compare les seuils.
- **La partie alerte locale/distante et stockage/affichage dans le tableau de bord :** un LED rouge, accompagnée d'un buzzer deviennent active lorsque le seuil fixé a été dépassé. Dans ce même contexte, le Wi-Fi de la carte ESP32 envoie des alertes grâce l'API de Telegram.

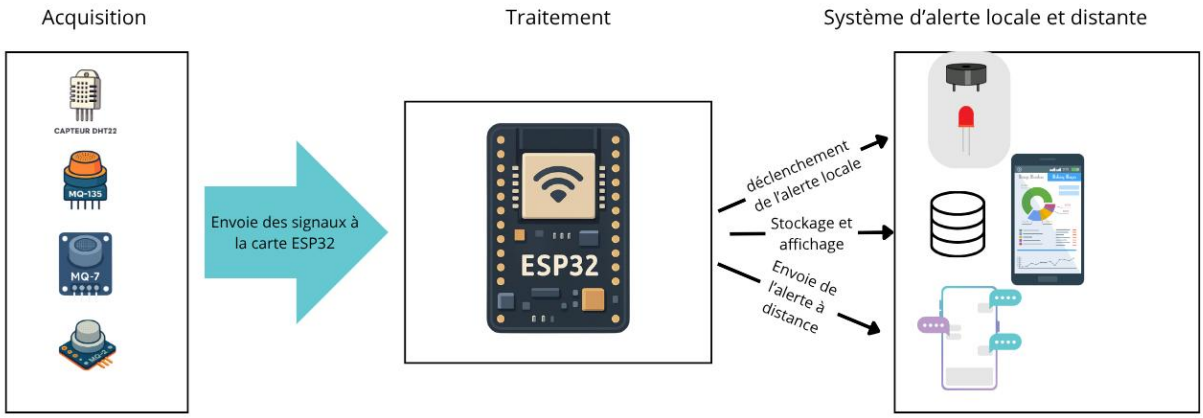


Figure 5 : Architecture générale du système

2.1.2 Diagramme de fonctionnement du système

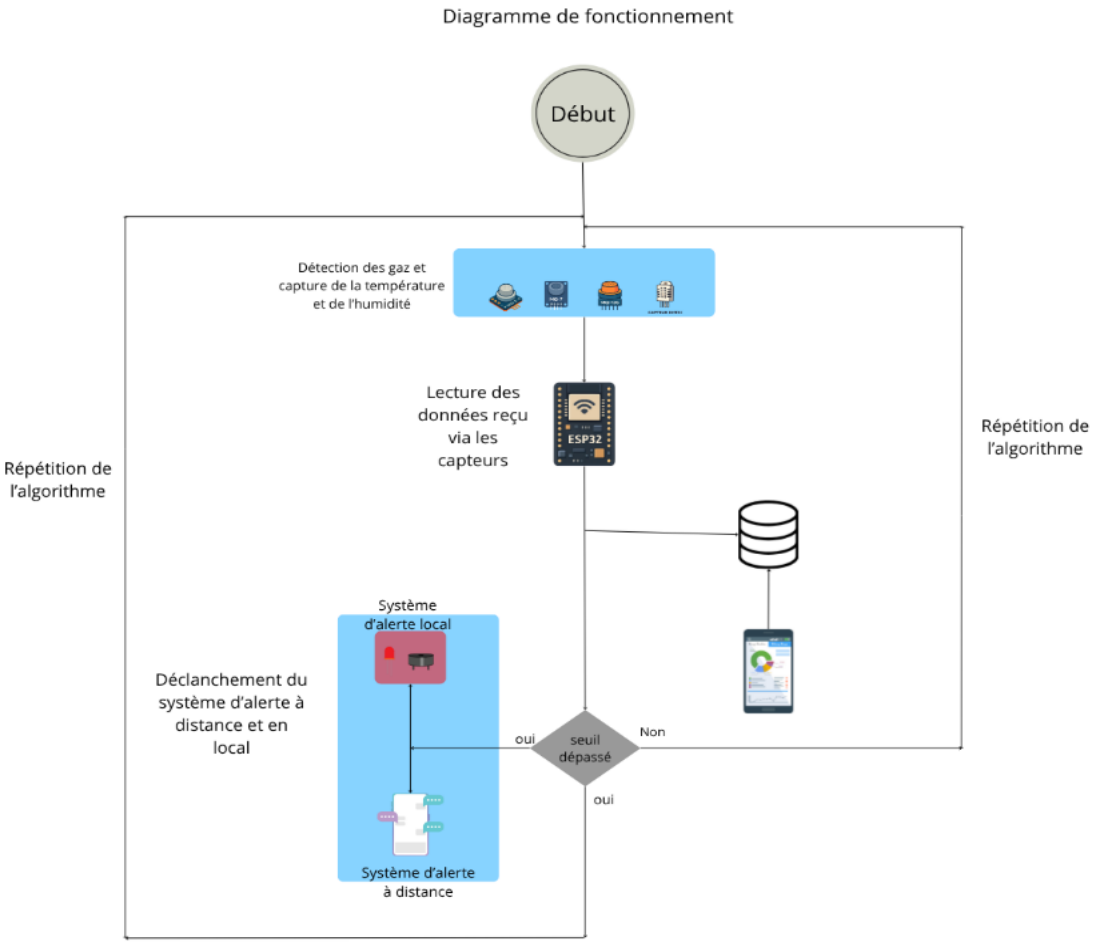


Figure 6: Diagramme de fonctionnement

Cette version de diagramme représente le résultat attendu de notre projet. Cependant, ce que l'on va retenir de ce diagramme sont les étapes suivantes :

- Dès la mise en marche de notre système, les capteurs de gaz et celui de l'humidité et la température entrent en action pour mesurer la concentration en gaz et de mesurer la température, l'humidité de l'air.
- Les données seront ensuite envoyées dans une base de données en ligne pour y être stocké puis renvoyé dans le tableau de bord pour le suivi en temps réel.
- Après l'étape précédente, la carte ESP32, lit les capteurs et corrige les données reçues avec les valeurs de l'humidité et la température et compare les données après correction avec les seuils fixés.
- Enfin, lorsque les mesures dépassent une des seuils, le système d'alerte s'actionne localement et par la Wi-Fi une notification Telegram sera envoyé vers l'utilisateur à distance, et reprend le processus dès le début. Sinon, le processus se réengage dans un nouveau processus.

2.1.3 Diagramme de classe

Le diagramme de classe présente la structure statique de notre système de surveillance de la qualité de l'air intérieur. Il décrit les principales classes, leurs attributs et les relations qui les lient, et met ainsi en évidence l'organisation logique des données et des traitements assurés par le prototype.

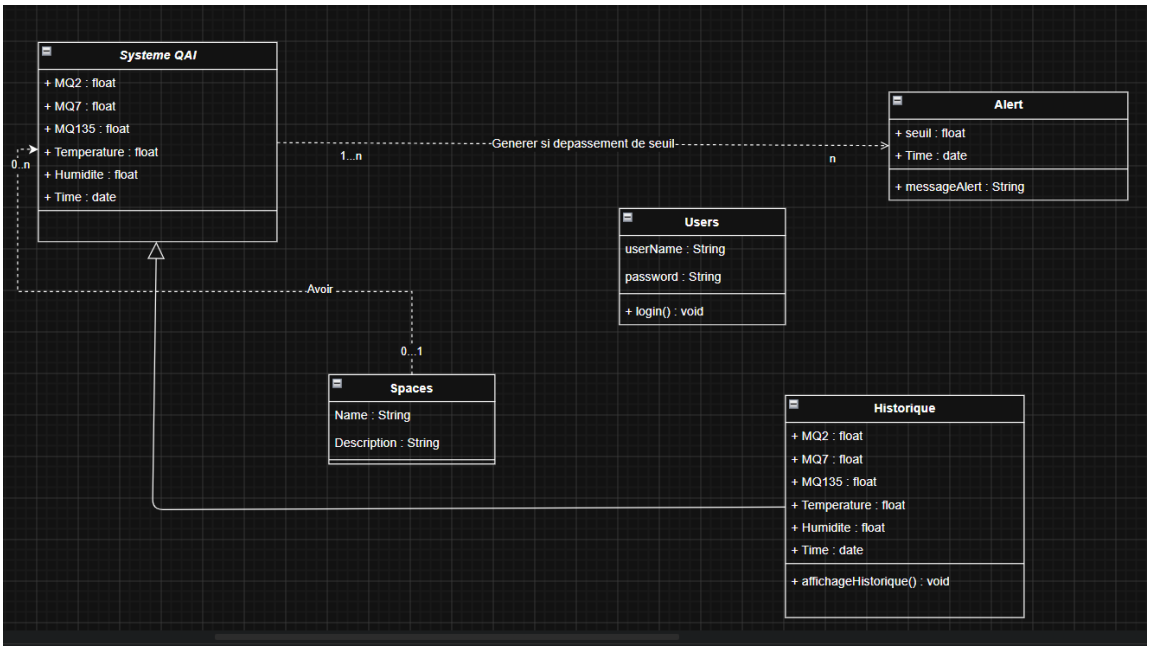


Figure 7 : Diagramme de classe

2.1.4 Diagramme de cas d'utilisation

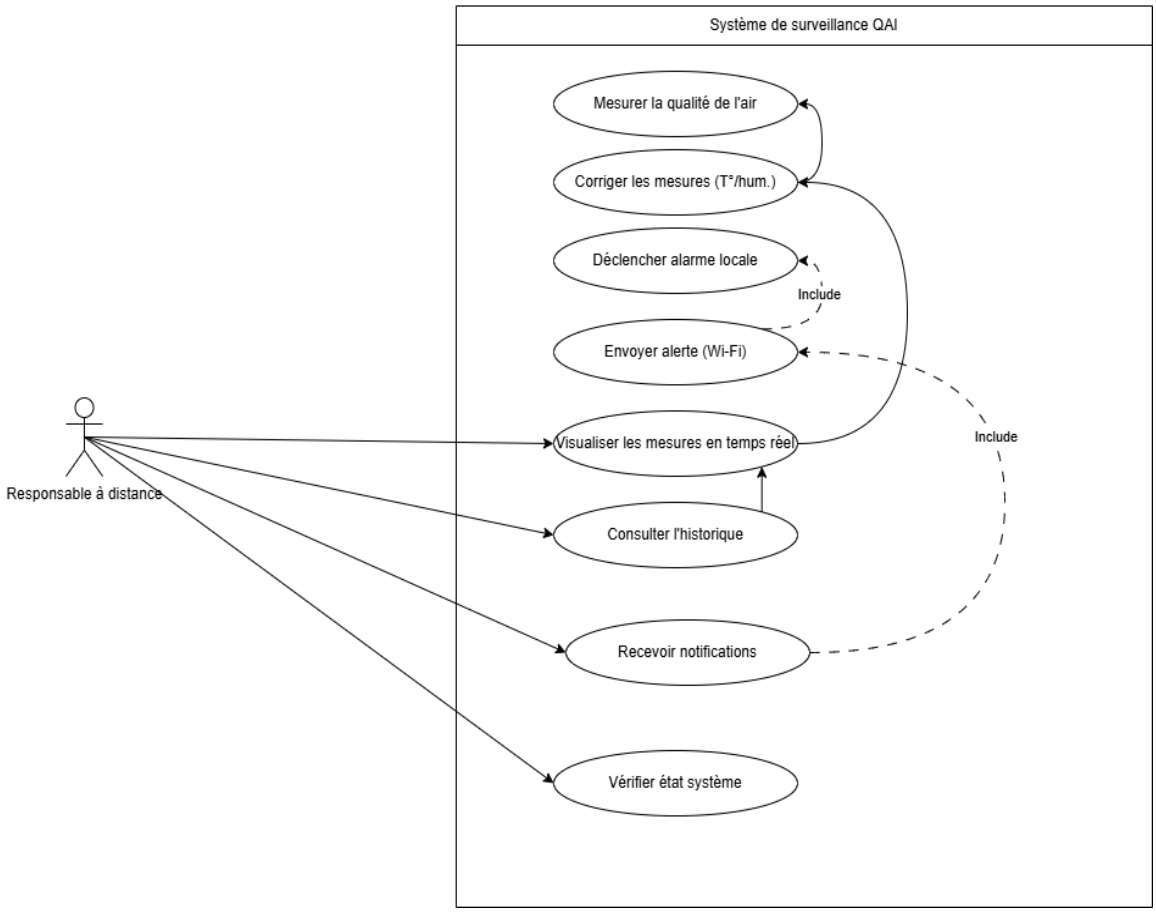


Figure 8: Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation représente les cas d'utilisation possible entre l'utilisateur du système et le système lui-même. Dans notre cas, ce diagramme de cas d'utilisation représente les cas d'utilisation entre un utilisateur distant avec notre système de surveillance de la qualité de l'air intérieur. Ce qui correspond à une version améliorée de notre travail.

Cependant nous allons nous contenter des cas d'utilisation suivante ;

- Mesurer la qualité de l'air (système)
- Corriger les mesures (système)
- Déclencher l'alarme locale (système)
- Envoie de l'alerte Wi-Fi (système)
- Recevoir les notifications (utilisateur distant)
- Vérifier l'état du système si active ou inactive (utilisateur distant)

2.1.5 Diagramme de séquence

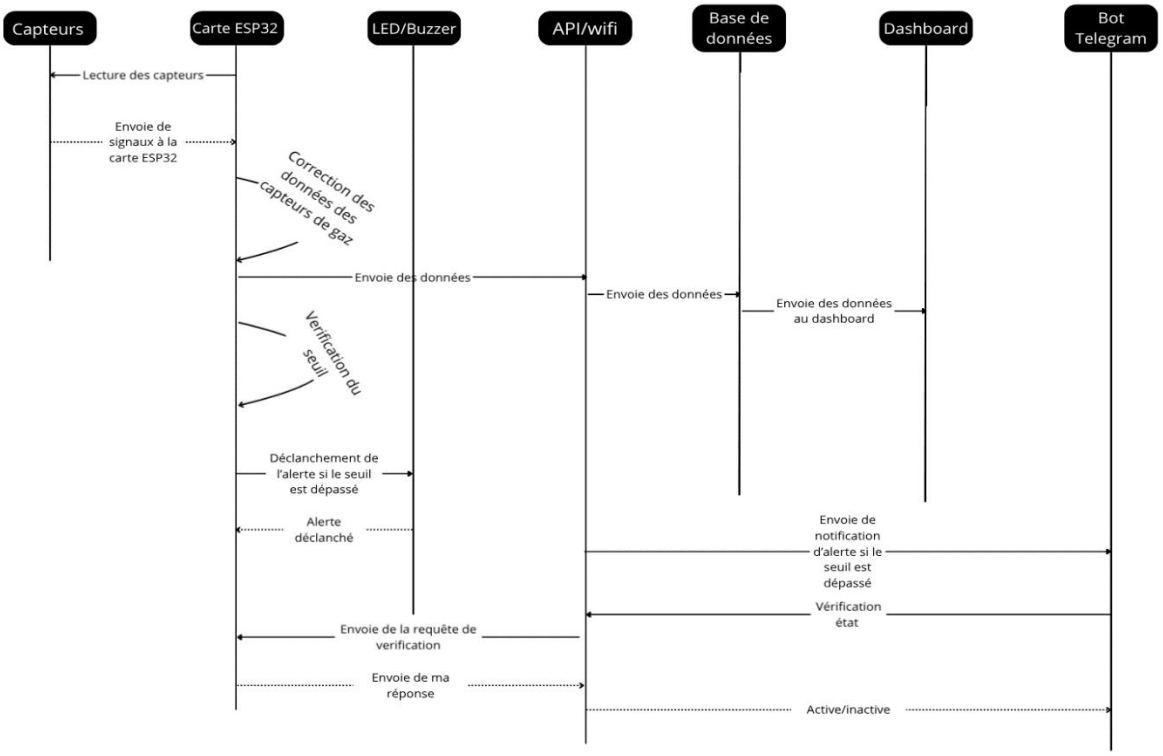


Figure 9 : Diagramme de séquence

Le diagramme de séquence détaille comment le système réalise une fonctionnalité.

Dans notre cas, nous pouvons observer les séquences suivantes :

- Lecture des capteurs par la carte ESP32
- Envoie de signaux (données de mesures sous forme de signal) depuis les capteurs vers la carte
- Correction des données de mesure envoyé par les capteurs par la carte ESP32
- Envoie des données corrigées vers la base de données
- Récupération des données depuis la base de données et affichage de ces dernières sur le dashboard
- Vérification du seuil (si dépassé = présence de danger)
- Déclenchement d'une alerte locale si le seuil est dépassé
- Alerte déclenchée (son du buzzer et clignotement d'un LED rouge)
- Envoie de données vers l'API
- L'API envoi les données reçues vers le bot Telegram (client distant pour le suivi de l'activité du système)

- Envoie d'une requête de vérification de l'état d'activité du système à partir du Bot
- L'API envoie la requête à la carte
- Réponse de la carte
- L'API envoie une réponse au bot Telegram (active ou inactive)

2.2 Réalisation du prototype

Dans la conception nous avons défini l'architecture du système et le comportement des parties prenantes face à ce dernier. Cette section détaille la concrétisation pratique de notre système. Nous adoptons une approche progressive en trois étapes :

- **Conception matérielle :** élaboration des schémas de câblage pour chaque composant avant l'assemblage final.
- **Intégration embarquée :** configuration et personnalisation du firmware développé en MicroPython sur la carte ESP32, cœur du projet.
- **Ecosystème logiciel :** présentation de la chaine d'outils (IDE, simulateur, …) ayant permis le développement, le test et le déploiement de la solution.

2.2.1 Conception matérielle

Dans cette section, nous allons montrer le branchement de tous les matériaux de notre système depuis les capteurs sensoriels, le capteur de température et de l'humidité, le système d'alerte locale, et le schéma du câblage complet du système.

2.2.1.1 Schéma de câblage des matériaux utilisés

Pour réaliser notre prototype, nous avons fait une sélection de matériels spécifique définis dans l'état de l'art. Nous allons montrer dans cette section le branchement de chaque composant ensuite montrer le rendu final voulu dans l'assemblage.

- **Les capteurs MQ :**

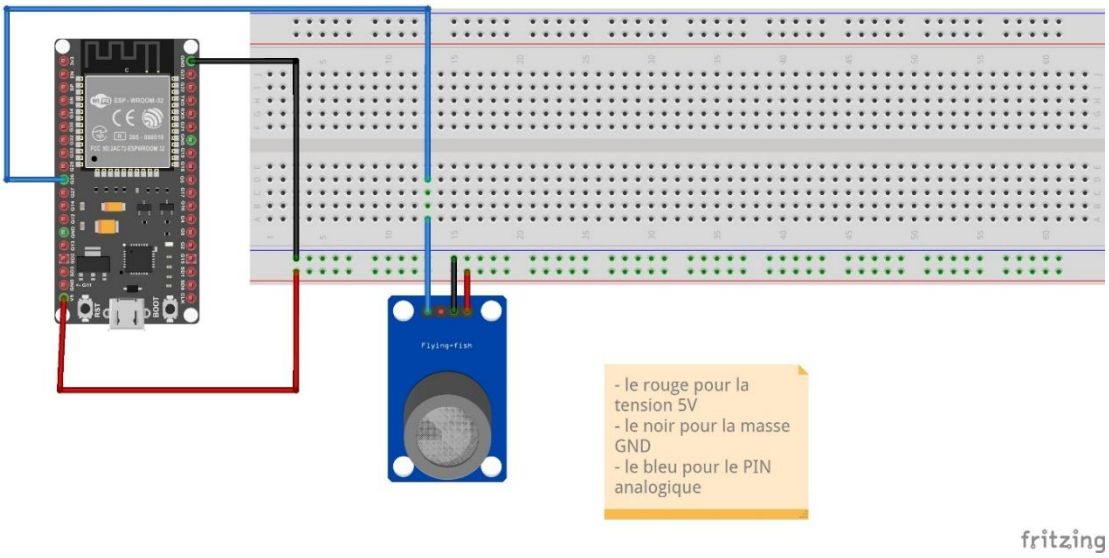


Figure 10: Schéma de branchement des capteurs MQ

Tous les capteurs de la famille MQ sont branchés de la même façon. Une broche pour l'alimentation, une autre pour la masse et une sortie analogique qui enverra des données analogiques à la carte ESP32.

- **Le capteur de température et d'humidité :**

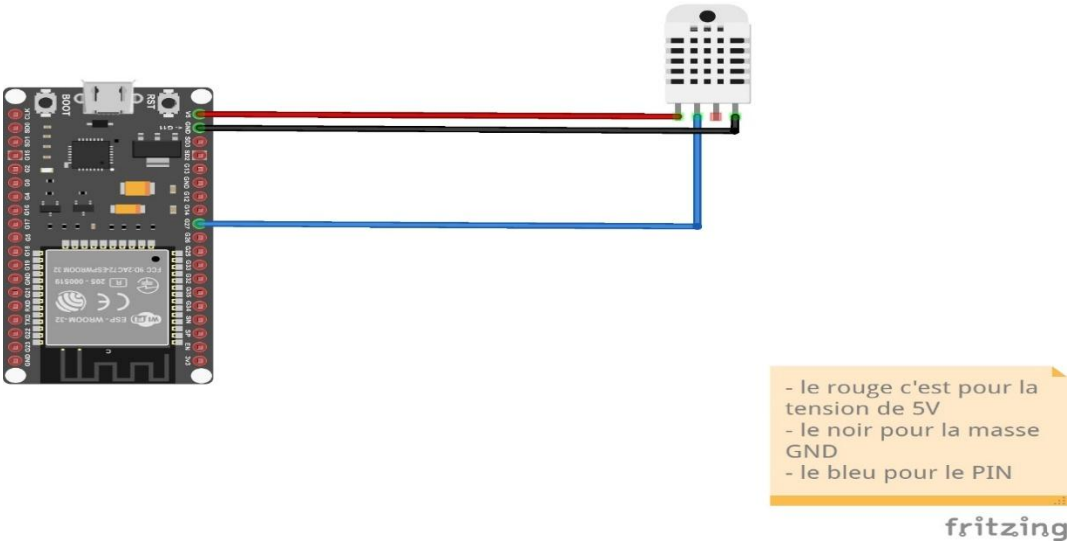


Figure 11 : Schéma de branchement du capteur DHT22

- **Le système d'alerte local :**

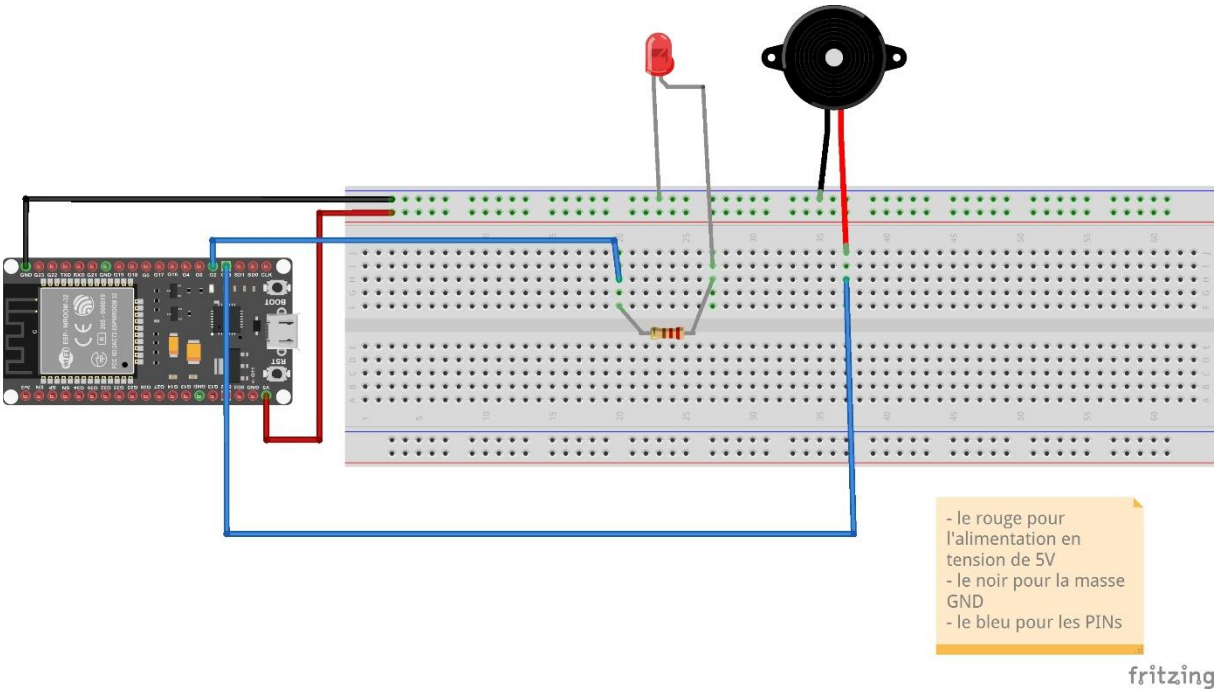


Figure 12 : Schéma de branchement du système d'alerte locale

Tout ce système suit une logique de branchement dans la carte suivante :

Tableau 2: Tableau de branchement des composants

Tableau de branchement		
Composants	**Broche ESP32**	**Alimentation**
MQ-2 (sortie analogique AO)	GPIO34	5V
MQ-7 (sortie analogique AO)	GPIO35	5V
MQ-135 (sortie analogique AO)	GPIO32	5V
DHT22	GPIO33 (data)	3.3V
LED rouge (anode)	GPIO26	3.3V (avec une résistance de 220 ohms)
Buzzer actif (signal)	GPIO27	5V

Ce tableau illustre le branchement des composant sur la carte ESP32. Les capteurs de gaz suscite une entrée/sortie analogique puisqu'ils renvoient des signaux analogiques.

2.2.1.2 Schéma du câblage final attendu

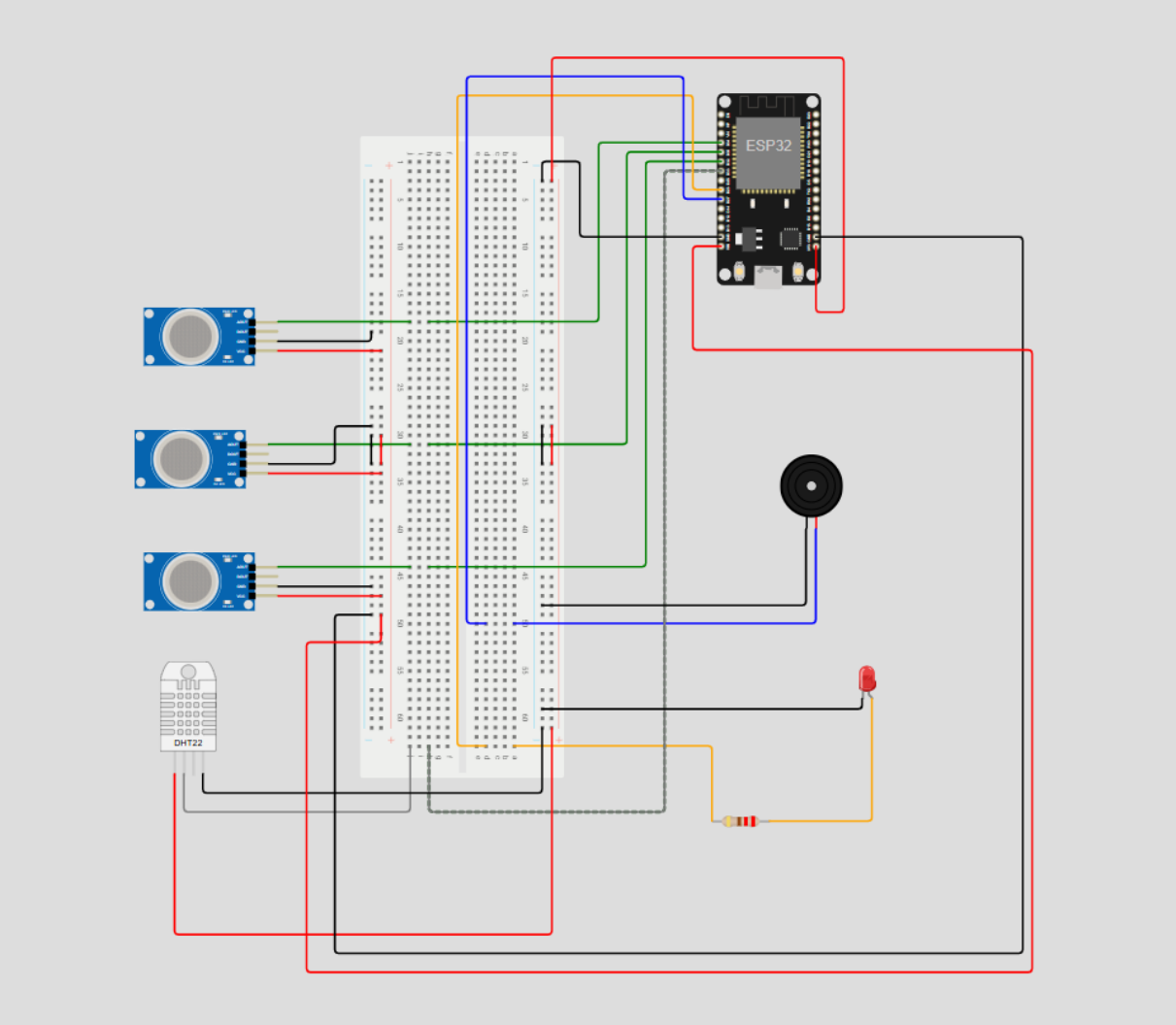


Figure 13: Schéma de branchement du système de la QAI

Ce schéma montre le branchement complet des composants dans la carte ESP32. A gauche nous avons les capteurs de gaz MQ et le capteur de température et d'humidité DHT22. Puis à droite nous avons le système d'alerte locale composée d'un buzzer et un LED rouge pour la signalisation.

2.2.2 Intégration embarquée

Dans l'intégration, nous montrons les configurations nécessaires pour mettre en place ce système. Dans le programme micropython, nous intégrerons les paramètres du wifi qui est crucial pour la communication du système et l'interface de visualisation et d'alerte, les paramètres du protocole de connexion et la configuration de l'interface de visualisation et d'alerte.

2.2.2.1 Paramètres du Wi-Fi

Communément appelé Wi-Fi, le Wireless Fidelity est notre support de transmission depuis la carte vers la base de données et vers l'interface de visualisation et d'alerte. Sa configuration se caractérise par son SSID ou le nom du point d'accès à la connexion Wi-Fi et de son mot de passe. La configuration se conforme aux images suivantes :

Tableau 3 : configuration du Wi-Fi

Configuration du Wi-Fi	
Définition des identifiants de connexion	**Fonction de connexion au Wi-Fi**

`# --- Identifiants WiFi ---`
`SSID = "Wokwi-GUEST"`
`PASSWORD = ""`
	`# Connexion WiFi`
	`# ============================================================`
	`def connecter_wifi():`
	`  wlan = network.WLAN(network.STA_IF)`
	`  wlan.active(True)`
	`  if not wlan.isconnected():`
	`  print("Connexion au WiFi :", SSID)`
	`  wlan.connect(SSID, PASSWORD)`
	`  while not wlan.isconnected():`
	`  print(".", end="")`
	`  time.sleep(0.5)`
	`  print("\n[OK] WiFi connecte ! IP :", wlan.ifconfig()[0])`

2.2.2.2 Configuration du protocole de communication

Pour la communication de la carte et de l'interface de visualisation et d'alerte nous avons le choix entre l'utilisation du protocole Web **http** ou bien le **MQTT** qui est aussi un protocole de communication beaucoup plus utilisé dans des projets IoT.

Nous avons choisi le protocole MQTT par sa légèreté, sa rapidité et sa sécurité. Le fonctionnement de ce protocole s'illustre comme suit :

Il fonctionne en suivant la logique « publisher/subscriber » (publicateur / abonné) ; notre carte ESP32 publie les données des capteurs dans un topic que l'on va qualifier de dossier où l'on va publier les données relatives à notre projet, ensuite, seul les subscribers ou abonnées à ce topic peuvent voir et avoir accès à ces données. Il n'y a pas de connexion directe entre l'interface de visualisation ou de notification et la carte, tout passe par l'intermédiaire d'un serveur central appelé Broker.

Tableau 4 : configuration du protocole de communication MQTT

Configuration du protocole de communication	
Définition des paramètres	**Fonction de connexion**

```
# --- Parametres MQTT EMQX ---
MQTT_SERVER = "n61222f2.ala.eu-central-1.emqxsl.com"
MQTT_PORT   = 8883                # port TLS/SSL
MQTT_USER   = "qai"
MQTT_PASS   = "qai2026@"
MQTT_CLIENT = "ESP32Client"
TOPIC       = b"esp32/airquality"
```

```
# ============================================================
#  Connexion / reconnexion au broker MQTT (TLS)
# ============================================================
def connecter_mqtt():
    global client
    # Contexte SSL : on accepte le certificat sans CA (test / Wokwi)
    contexte = ssl.SSLContext(ssl.PROTOCOL_TLS_CLIENT)
    contexte.verify_mode = ssl.CERT_NONE

    client = MQTTClient(
        MQTT_CLIENT, MQTT_SERVER, port=MQTT_PORT,
        user=MQTT_USER, password=MQTT_PASS,
        keepalive=60, ssl=contexte,
    )
    print("Connexion au broker MQTT EMQX ...", end="")
    client.connect()
    print(" [OK] Connecte a EMQX !")
```

Dans notre cas nous avons créé un broker dont l'adresse du serveur est **« n61222f2.ala.eu-central-1.emqxsl.com ».** Les données sont donc publiées dans le topic **« esp32/airquality »** seul les abonnées à ce topic peuvent recevoir les données que la carte publie. Pour renforcer la sécurité, nous avons créé ainsi l'utilisateur **« qai »** authentifié par **« qai2026@ »** pour pouvoir utiliser les paramètres TLS/SSL du broker. Dans notre cas nous sommes surs que les données publié dans ce Broker personne ne peut avoir accès s'il n'est pas abonné au topic **« esp32/airquality »**.

2.2.2.3 Configuration de l'interface de visualisation et d'alerte

Pour notre projet, nous avons utilisé une plateforme web qu'on a nommé AirWatch pour visualiser nos données qui serons stockées dans une base de données MySQL.

En effet, nous avons d'abord créé la base de données **« airquality »,** ensuite nous avons connecté la plateforme à la base de données pour lire les données depuis notre base de données MySQL puis les interpréter dans un tableau de bord.

Le processus se fait suivant les illustrations suivantes :

- Création de la base de données : dans notre cas, puisque nous avons utilisé docker pour le déploiement de notre projet, nous allons donc créer la base de données via

l'interpréteur de commande de docker après s'être connecté. Nous pouvons aussi utiliser un client MySQL pour nous connecter puis ensuite créer la base de données.

```
PS C:\QAI> docker exec -it mysql mysql -u user -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 13
Server version: 8.0.46 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
```

Figure 14 : Console powershell dans docker / connexion à la base de données MySQL

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| airquality         |
| information_schema |
| performance_schema |
+--------------------+
```

Figure 15: console mysql

- Ensuite nous allons créer la table qui recevra les données publiées dans le topic défini précédemment :

```
mysql> describe airquality;
+-------+-----------+------+-----+-------------------+-------------------+
| Field | Type      | Null | Key | Default           | Extra             |
+-------+-----------+------+-----+-------------------+-------------------+
| id    | int       | NO   | PRI | NULL              | auto_increment    |
| mq2   | int       | YES  |     | NULL              |                   |
| mq7   | int       | YES  |     | NULL              |                   |
| mq135 | int       | YES  |     | NULL              |                   |
| temp  | float     | YES  |     | NULL              |                   |
| hum   | float     | YES  |     | NULL              |                   |
```

Figure 16: Console mysql (description de la table airquality)

- Puis nous avons créé la plateforme AirWatch pour le suivi en temps réel dont elle est composée des interfaces suivantes :
    - Le Dashboard : l'interface qui affiche l'évolution des données au cours du temps avec un auto rafraichissement de 5 secondes ; dans cette interface on retrouve les courbes de niveau des données issues de tous les capteurs ainsi que leurs valeurs et ces derniers changent de couleur en fonction de la valeur reçue en comparaison avec le seuil défini.

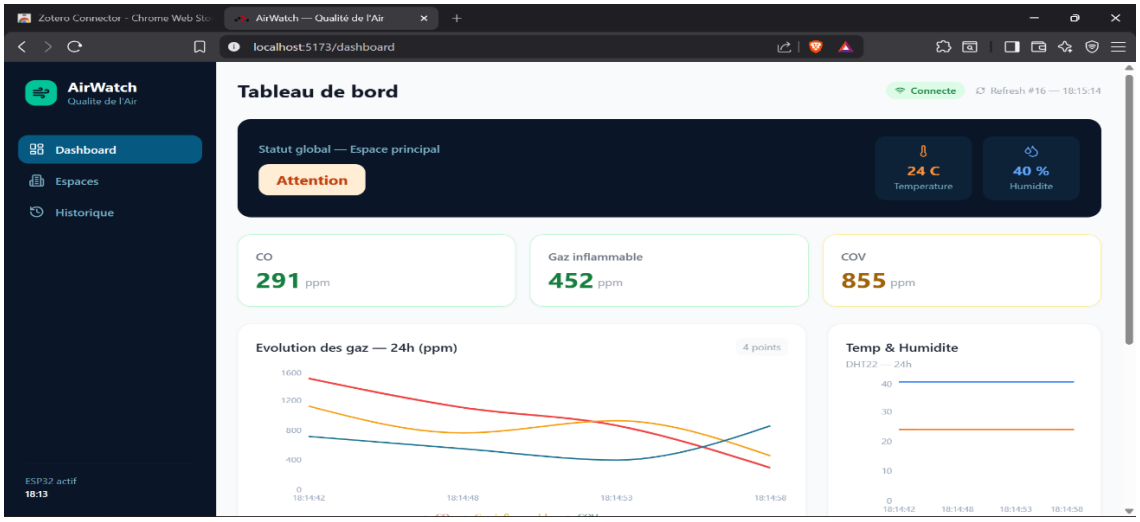


Figure 17: Interface du tableau de bord AirWatch

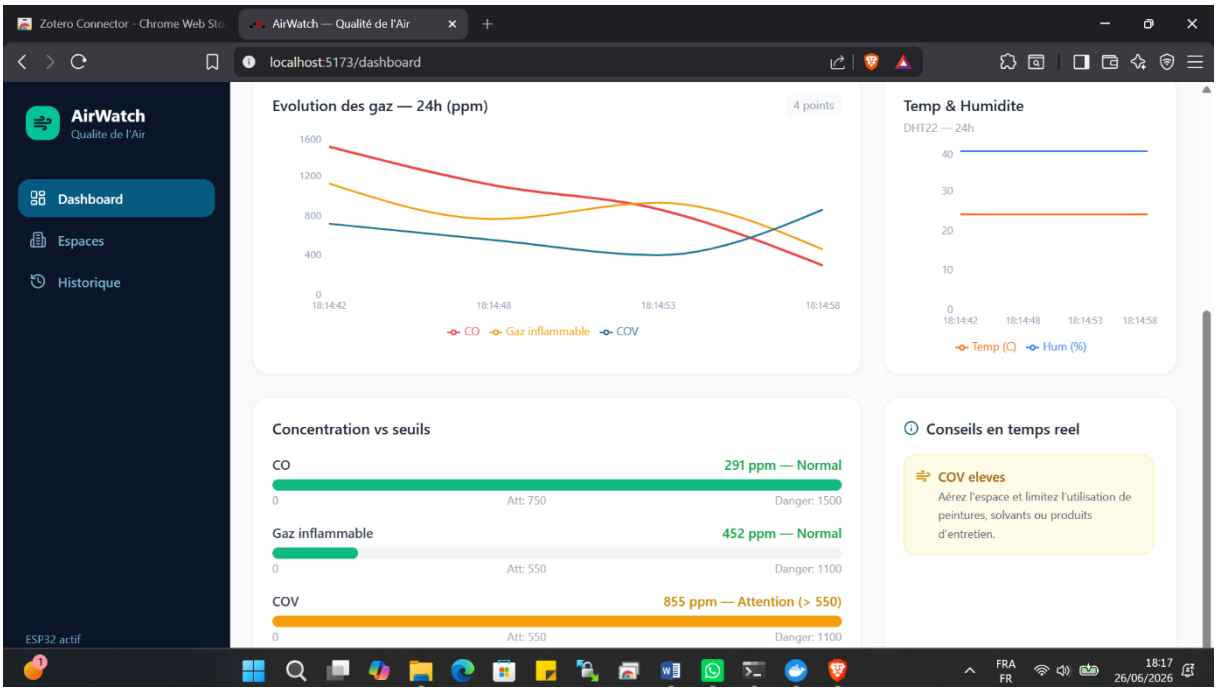


Figure 18: Interface du tableau de bord AirWatch 2

- Puis on a l'interface Espaces, qui regroupent la ou les salles où est disposé le dispositif de surveillance sachant que l'on peut communiquer plusieurs dispositifs avec la plateforme.

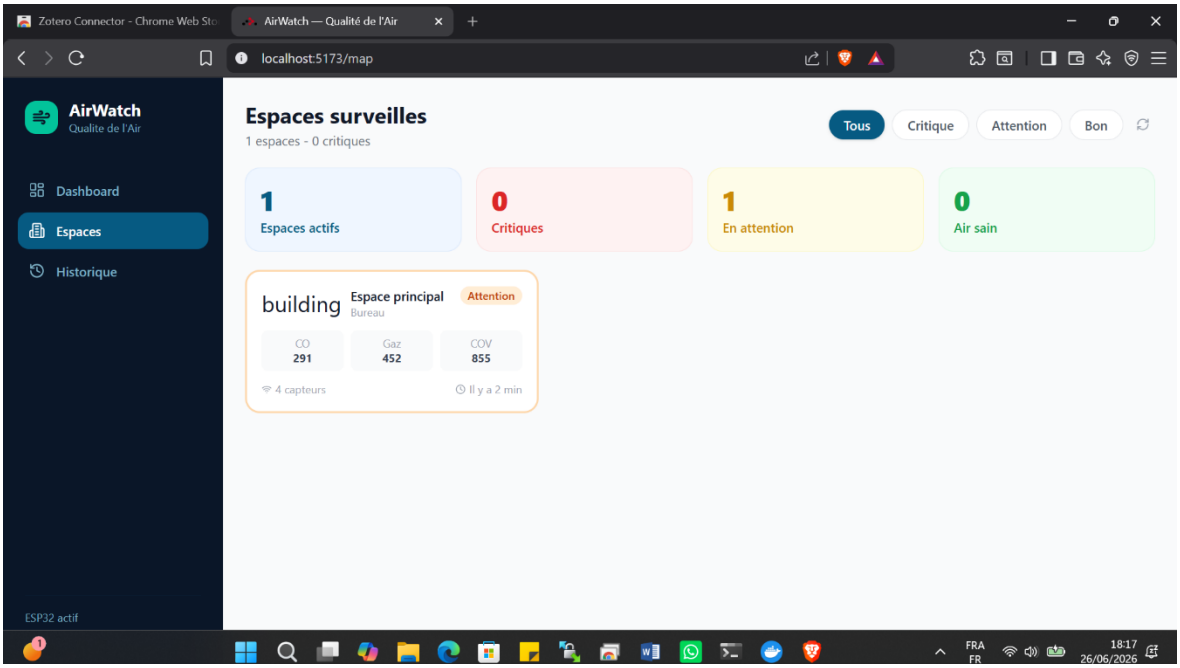


Figure 19: Interface du tableau de bord AirWatch partie Espaces surveillés

- Enfin l'interface historique qui contient l'historique des données reçues depuis la mise en marche du système jusqu'à l'arrêt. Il y a également la possibilité de télécharger cette historique en format CSV en cas de besoin pour des éventuelles études.

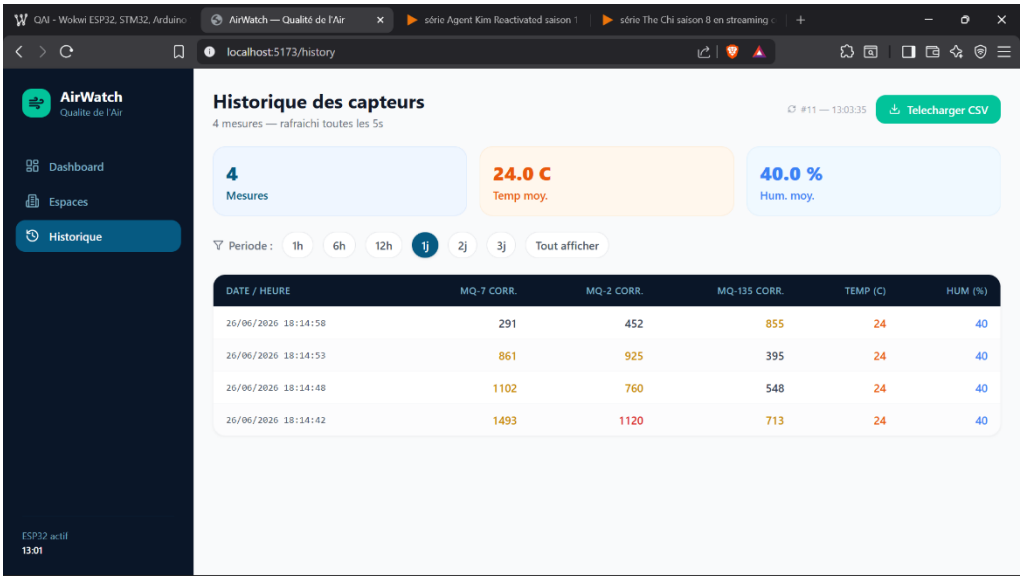


Figure 20 : Interface du tableau de bord AirWatch partie Historique

Notre plateforme joue le rôle de moniteur de suivi. Elle est inspirée des diagrammes de cas d'utilisation et de séquence précédentes.

Pour l'interface d'alerte nous avons utiliser telegram pour recevoir les notifications. De ce fait nous allons d'abord configurer notre client télégram qui récevra les notification. Le processus s'éexecute comme suit :

- Nous allons d'abord créer un Bot qui sera notre client télégram pour recevoir les notifications : pour cela on tape **Botfather** dans la barre de recherche, c'est un assistant mis en place par télégram pour nous servir d'interpréteur de commande à la création de notre client ;

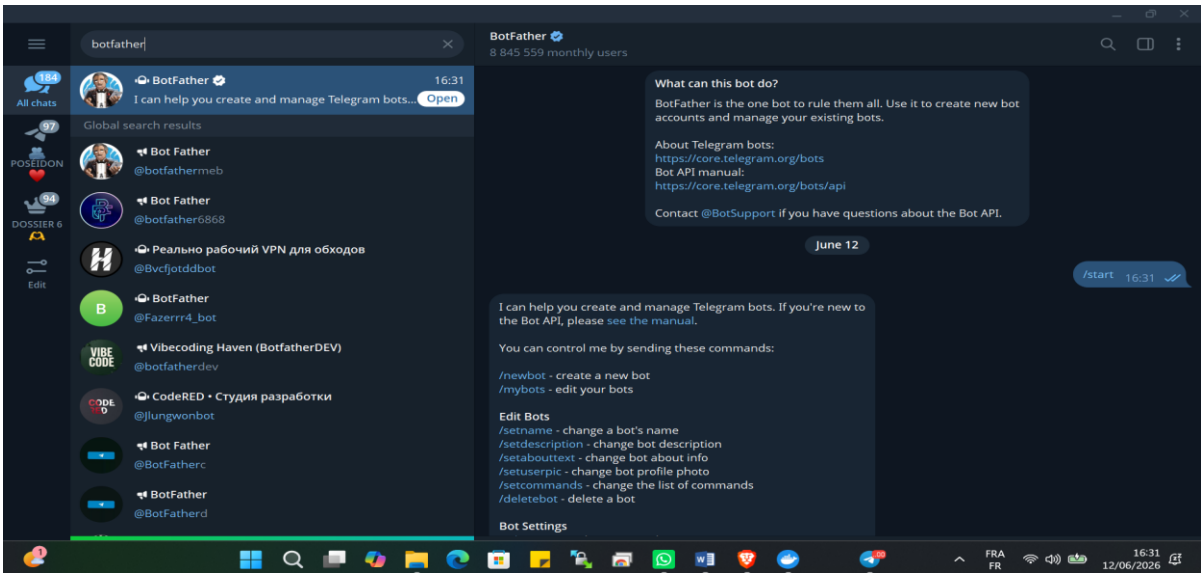


Figure 21 : Interface de création du bot Telegram (Botfather)

Ensuite nous suivons les étapes de la création du Bot en exécutant la commande de création indiqué « **/newbot** » ; puis on choisit le nom de notre client ; ensuite le nom d'utilisateur de notre bot dont par cette discussion télégram nous recevrons les alertes ; ensuite on nous listera les informations de notre client ainsi que la clé pour avoir accès à notre API télégram ;

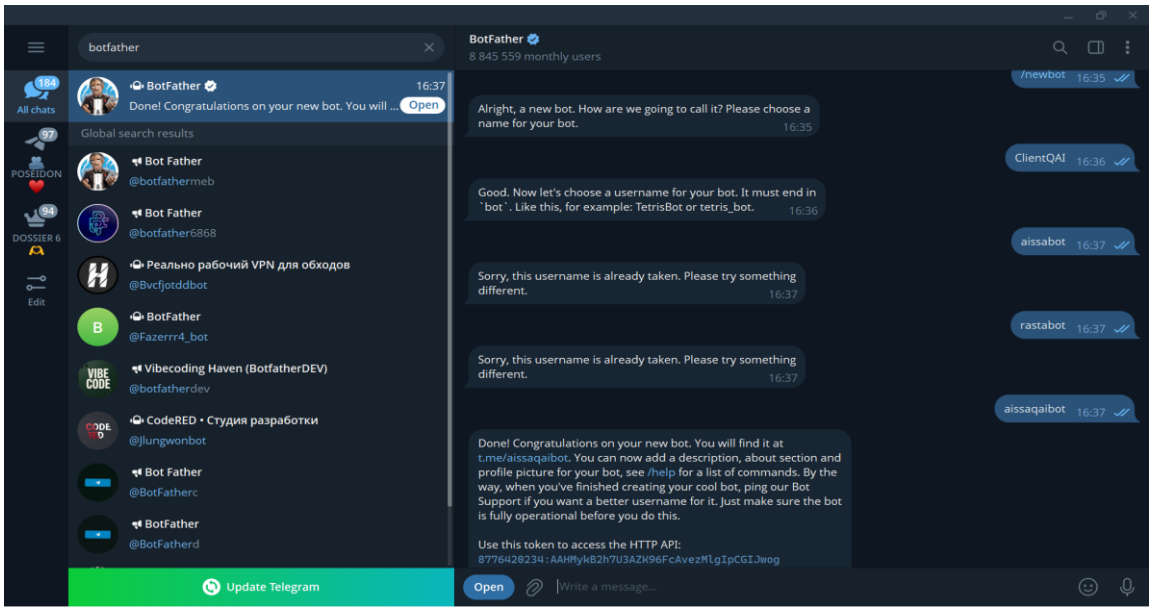


Figure 22 : Interface de création du bot Telegram (Botfather) finalisation

Voici notre client télégram que nous avons créé et par lequel nous allons recevoir les alertes en cas de dépassement de seuil.

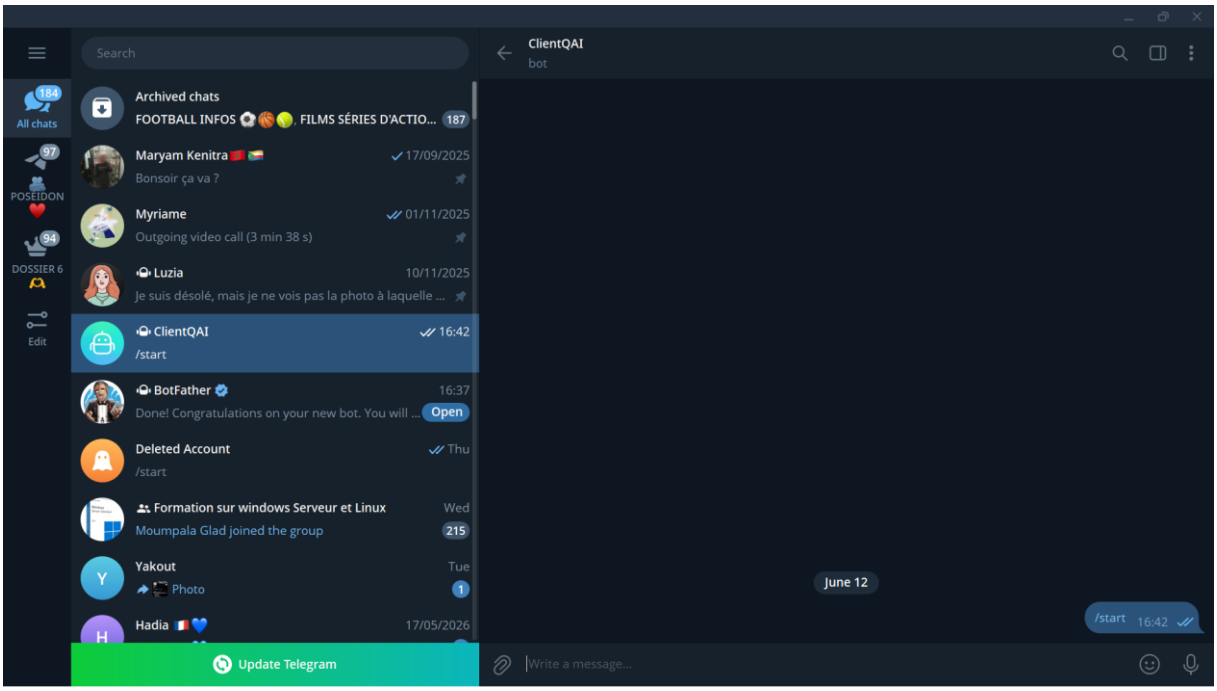


Figure 23 : Interface de discussion du bot Telegram

Pour finir, nous allons intégrer l'API de ce client dans notre programme pour qu'il puisse recevoir les notifications.

De ce fait, on définit les variables **TOKEN** qui contiendra la clé pour communiquer avec l'API télégram et l'identifiant du chat.

Ensuite on définit la fonction qui nous permettra d'envoyer des messages dans la carte ESP32:

```
59    def send_telegram_message(msg):
60        url = "https://api.telegram.org/bot{}/sendMessage".format(TOKEN)
61        data = {"chat_id": CHAT_ID, "text": msg}
62        try:
63            r = urequests.post(url, json=data)
64            print("Telegram:", r.text)
65            r.close()
66        except Exception as e:
67            print("Erreur Telegram:", e)
```

Figure 24 : Définition de la fonction d'envoi du message Télégram

Après cette étape, on peut envoyer des messages d'alertes vers notre bot télégram en cas de dépassement de seuil :

```
164    seuils = {
165        "mq2": 1100,
166        "mq7": 1500,
167        "mq135": 1100
168    }
169    alerte = any(donnees[capteur] > seuil for capteur, seuil in seuils.items())
170    buzzer.value(0)
171    led.value(0)
172
173    if alerte:
174        print(">>> ALERTE : seuil de gaz depasse ! <<<")
175        for capteur, seuil in seuils.items():
176            if donnees[capteur] > seuil:
177                send_telegram_message(f">>> ALERTE : seuil de gaz depasse ! <<<\n{capteur.upper()} : {donnees[capteur]} ppm")
178        led.value(1)
179        buzzer.value(1)
180        time.sleep(0.1)
```

Figure 25: Définition et contrôle des seuils

2.2.3 Ecosystème logiciel

Nous avons utilisé une multitude de logiciels, d'outils et de langage de programmation, afin d'arriver à un prototype minimum viable. Parmi eux nous avons :

Canva

Canva est un logiciel de design graphique. Nous l'avons utilisé pour effectuer nos diagrammes de fonctionnement pour donner une aperçue plus ou moins concrète du fonctionnement du système, le diagramme de séquence et le schéma en bloc du système.



Figure 26: Logo Canva

Fritzing

Fritzing est un logiciel libre de conception électronique. Avec ce logiciel nous avons conçu le schéma de câblage des capteurs et autres composants dans la carte ESP32. Ce logiciel est spécialement conçu pour les débutants, ce qui constitue un choix favorable pour nous.

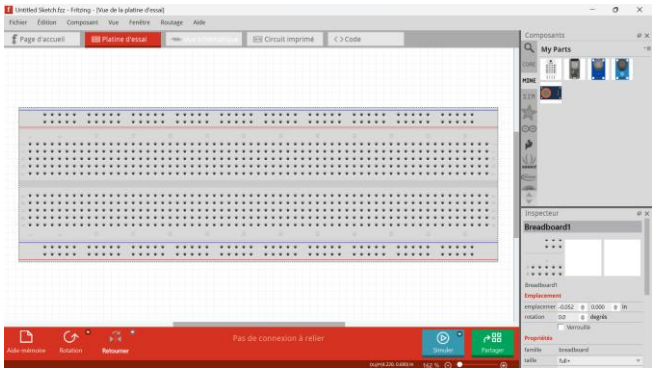


Figure 27 : Espace de travail de Fritzing

Visual Studio Code

Visual Studio Code est un éditeur de texte libre et open source. C'est l'un des environnements de développement les plus populaires. Avec son extensibilité et adaptabilité aux projets complexe, c'est une bonne alternative pour le développement de notre prototype.



Figure 28: Logo Visual Studio Code

Le MicroPython

Le MicroPython est une implémentation légère et optimisée du langage Python. Il est conçu spécifiquement pour fonctionner sur des microcontrôleurs et des systèmes embarqués. Il est conçu également pour fonctionner directement sur le matériel. Un choix judicieux pour la programmation de notre carte électronique ESP32.

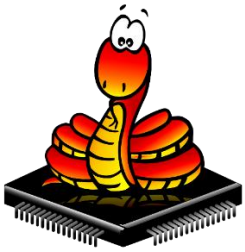


Figure 29 : Logo micropython

- **Quelques frameworks pour le développement de la plateform**

Notre plateforme se divise en 2 parties : le frontend et le backend.

- En frontend : la partie visible de l'utilisateur nous avons utilisé :React et vite, Tailwind CSS pour la mise en forme, recharts pour les graphiques, lucide react pour les icones, react router pour la navigation et socket.io client pour la réception des données en temps réel du backend.
- En backend : la partie qui interagi avec la base de données et les serveurs, nous avons utilisé : Node.js et Express pour leur rapidité, MQTT pour la communication avec la carte ESP32, MySQL2 pour la base de données, socket.io pour la transmission en temps réel vers le frontend, Telegram Bot API pour l'envoi des notifications d'alertes à notre client Telegram et dotenv pour le chargement des variables d'environnement depuis le fichier env de notre projet, cors pour autoriser le frontend à communiquer avec le backend en vue de leur origine différente et nodemon pour la surveillance des fichiers du backend en cas de modification.

- **La plateforme draw.io**

La plateforme draw.io est un logiciel de création de diagrammes gratuit, open-source et multiplateforme. Nous avons utilisé la version web avec un navigateur pour concevoir notre diagramme de cas d'utilisation.

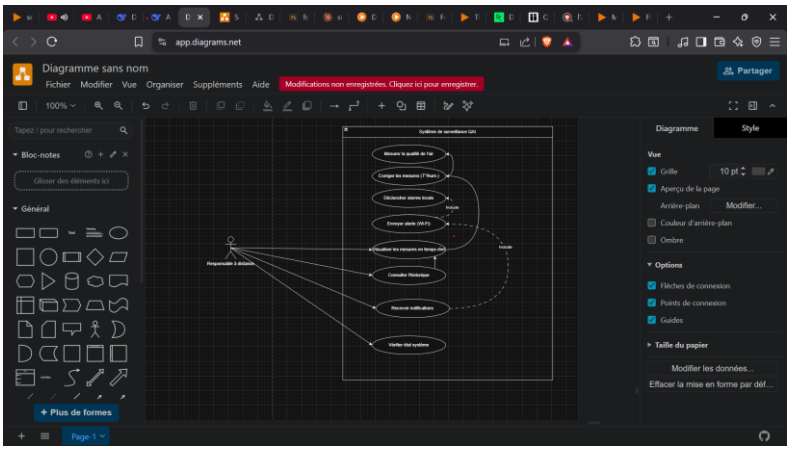


Figure 30: Interface de travail de la plateforme draw.io

Wokwi

Wokwi est une plateforme en ligne de simulation électronique. Elle nous a permis à réaliser nos tests directement dans un navigateur web sans avoir recours à des matériaux physiques.

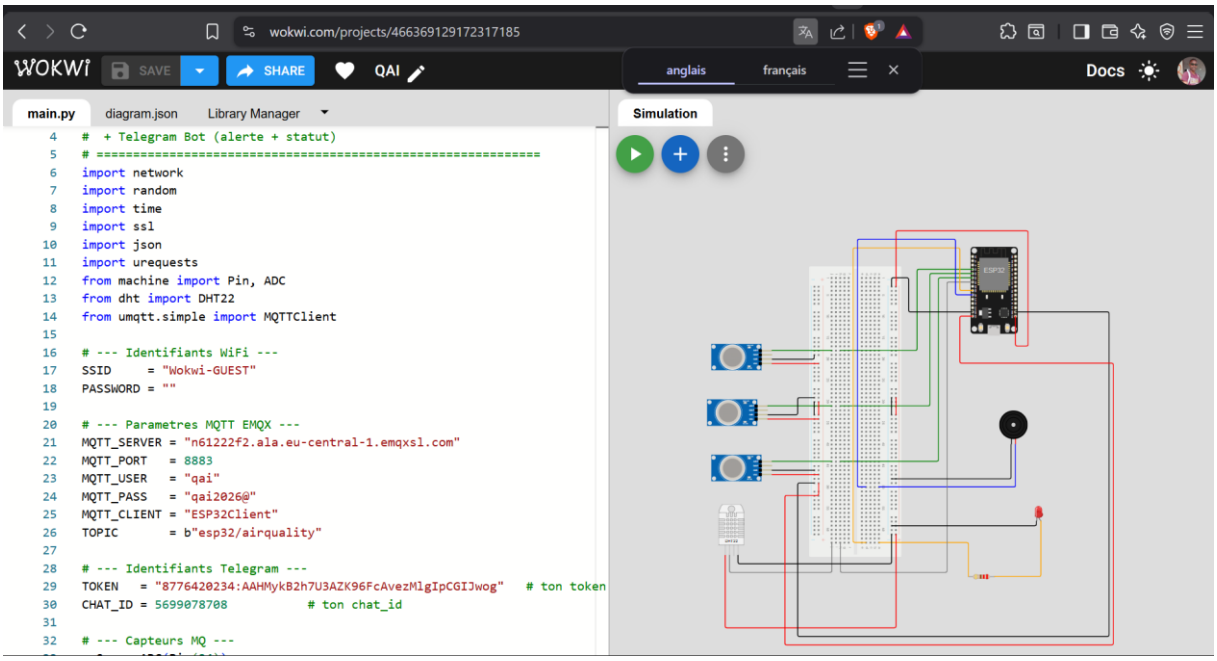


Figure 31 : Espace de travail de Wokwi

Node-RED

Node-RED est une plateforme de programmation visuelle conçue pour connecter facilement des appareils, des API et des services entre eux. Elle nous a été utile lors de la connexion de la carte ESP32 à la base de données ainsi qu'au protocole MQTT. A part le code

en micropython, Node-RED nous a permis de connecter facilement notre système à une base de données en ligne.

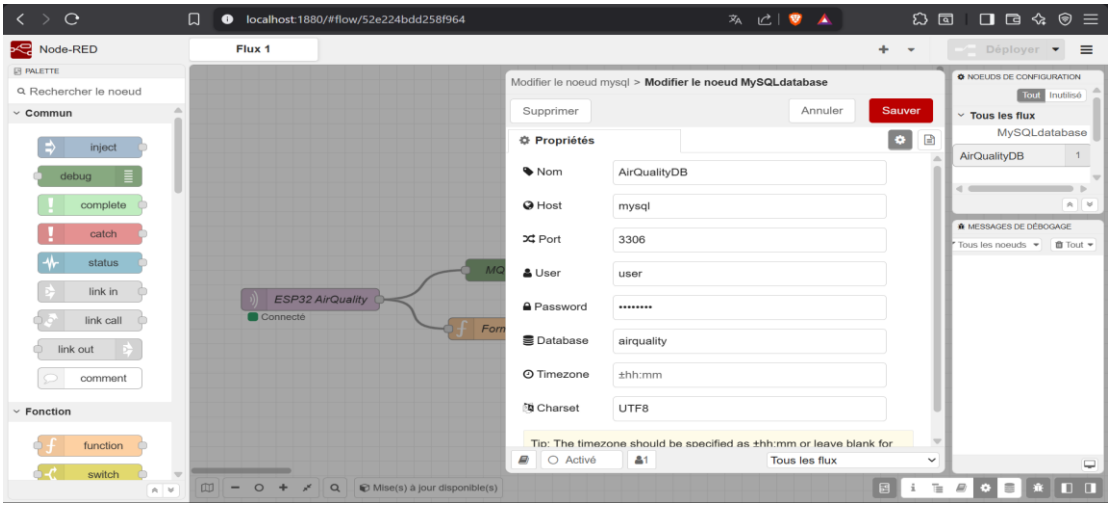


Figure 32 : Espace de travail de Node-RED

Docker desktop

C'est la version logicielle d'une plateforme de virtualisation légère qui permet de lancer des applications et services dans des environnements isolés appelés **conteneurs.** Il nous a permis à réunir tous les services nécessaires pour notre projet dans un seul conteneur.

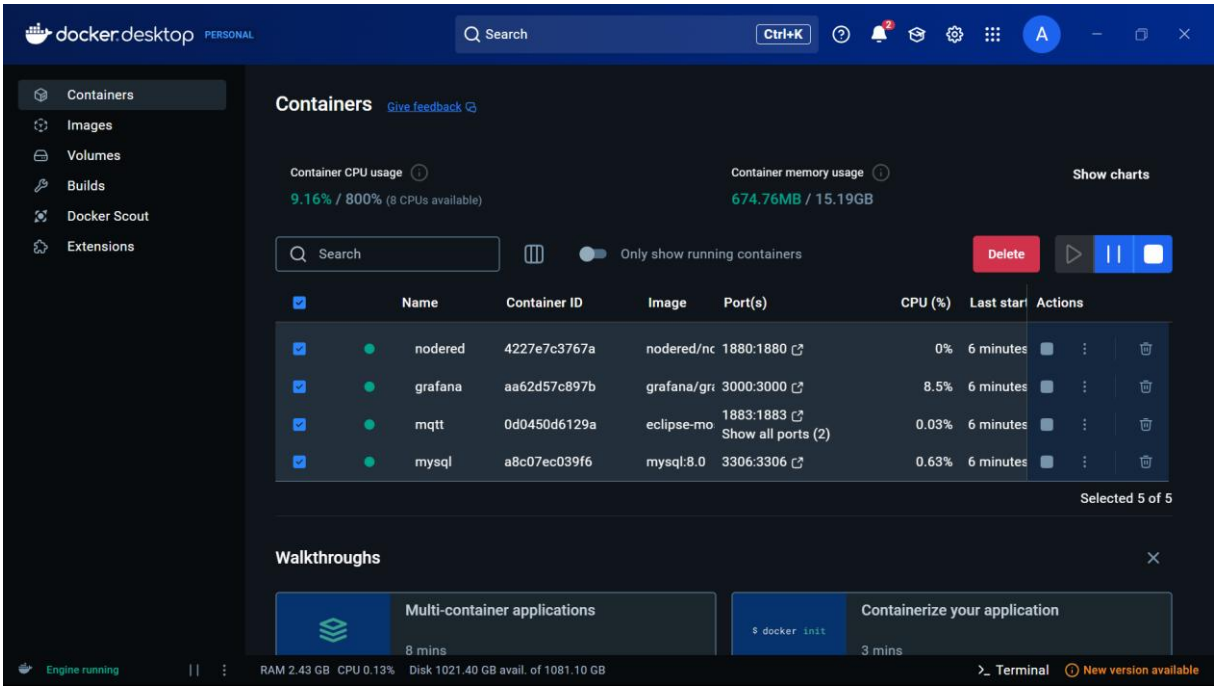


Figure 33 : Interface de Docker

Conclusion du chapitre 2

Dans cette partie nous avons montré en large la conception du système jusqu'à sa réalisation ainsi qu'à l'intégration du logiciel embarqué. Nous avons utilisé des outils de conception et de programmation pour mettre en place ce système. On a choisi d'utiliser une plateforme de simulation du résultat final pour des raisons de manque de temps en vue de la livraison des matériaux sur place. Toutefois, la réalisation reste possible avec la liste des matériaux précités dans ce document.

Chapitre 3 : Résultats et test de validation du prototype

Introduction

Après la conception et la réalisation de notre prototype, nous avons obtenu comme résultat un prototype de système de surveillance de la qualité de l'air intérieur. Nous avons procédé par la suite des tests afin de valider notre prototype et obtenir un prototype fiable. Dans cette section nous allons expliquer les étapes de tests effectué pour avoir ce rendu final qui est le prototype.

3.1 Scénario de test

Nous avons choisi un environnement virtuel pour simuler les tests de validation de notre système dans des conditions similaires à la réalité, en attribuant des valeurs approximatives. Pendant ces travaux, nous avons tout d'abord défini un intervalle de valeur à attribuer aux capteurs de gaz, puisque dans ce même environnement, les capteurs ne réagissent pas de façon dynamique. Nous avons donc appliqué une fonction de renvoie de valeurs aléatoires **random()** pour simuler les changements de valeur des capteurs. Pour le capteur de température et d'humidité, son comportement dans l'environnement de tests est dynamique puisqu'il applique le changement qu'on effectue pendant l'exécution du processus de simulation. Dans le programme nous avons ajouté le facteur de correction des valeurs des capteurs de gaz, pour avoir des valeurs tendant vers des conditions réelles d'utilisation de notre système.

A l'exécution, la carte ESP32 se connecte d'abord au Wi-Fi configuré dans le programme dont nous avons choisi d'utiliser le Wi-Fi propre à la plateforme wokwi nommé « **Wokwi-GUEST** ». Ensuite vient la connexion du Broker pour le protocole MQTT. Une fois connecté, le programme se met en marche et les données sont capturées, envoyées dans la base de données MySQL créée, puis renvoyé dans le tableau de bord Grafana pour y être affichée et permettre à l'utilisateur de suivre en temps réel la variation de la température et l'humidité et la présence de gaz dans la pièce.

La température et l'humidité influencent les données des capteurs de gaz. De ce fait, les valeurs de ces derniers varient en fonction de la température et l'humidité de la pièce. Les données capturées à 25 degrés Celsius et à 50% d'humidité dans la pièce ne seront pas les mêmes que les données capturées dans une pièce à 50 degrés Celsius de température et 70% d'humidité. Une fois le seuil dépassé, le système d'alerte local se met en marche en émettant un son produit par le buzzer et l'allumage d'une diode électroluminescent rouge et une notification est envoyé à notre bot Télégram configuré avec la valeur du capteur ayant dépassé le seuil de présence de gaz.

3.2 Résultat obtenu

Les résultats obtenus après les tests s'illustrent dans les images suivantes et tableau suivant :

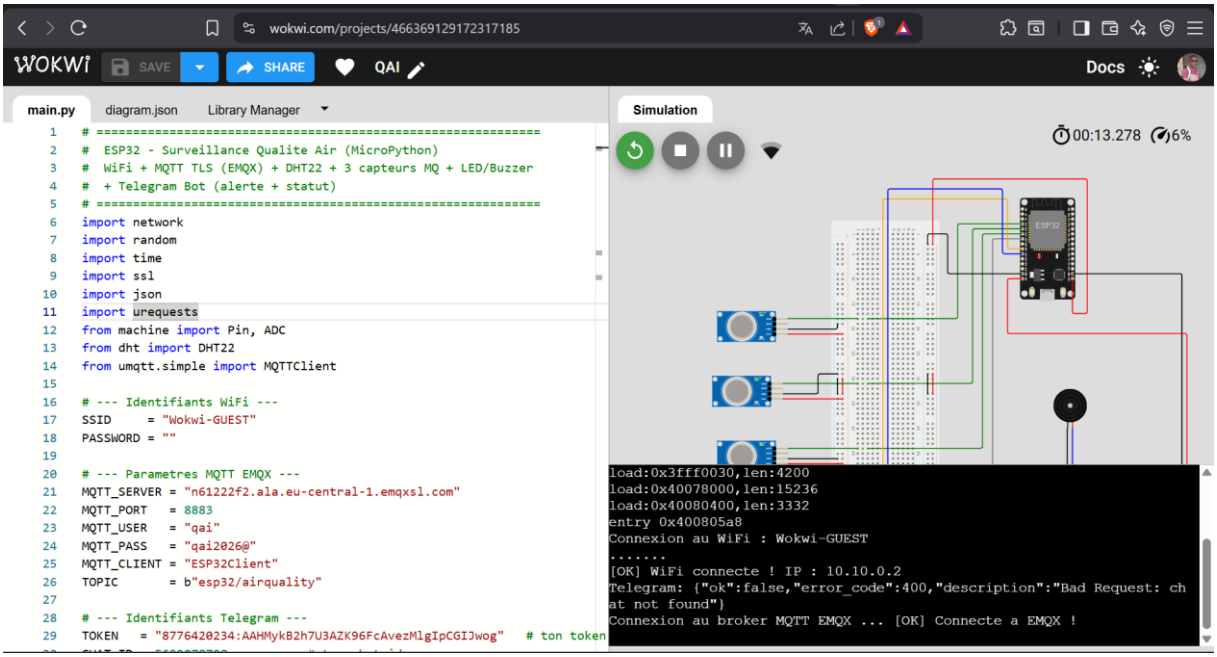


Figure 34 : Interface de Wokwi pendant un premier test

Dans cette image, nous pouvons observer la connectivité du système dans le Wi-Fi de la plateforme wokwi dans la console en bas à droite de l'image ainsi que la connectivité au broker MQTT. Il y a aussi le symbole de Wi-Fi qui l'indique dans l'image.

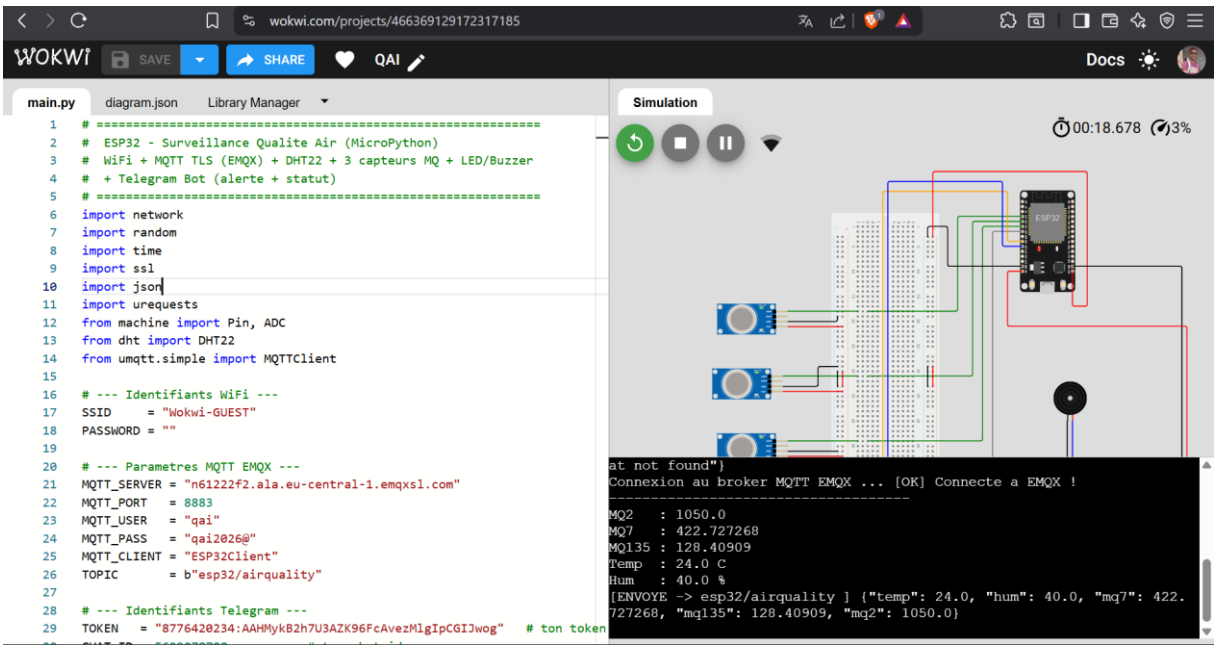


Figure 35 : Interface de Wokwi pendant un second test

Comme nous pouvons l'observer dans cette image, nous avons reçu les données des capteurs présents dans notre système qui indiquent :

- MQ2 : 1050.0 ppm
- MQ7 : 422.727268 ppm
- MQ135 : 128.40909 ppm
- Température : 24.0 degré
- Humidité : 40%

Maintenant appliquons une variation de la température :

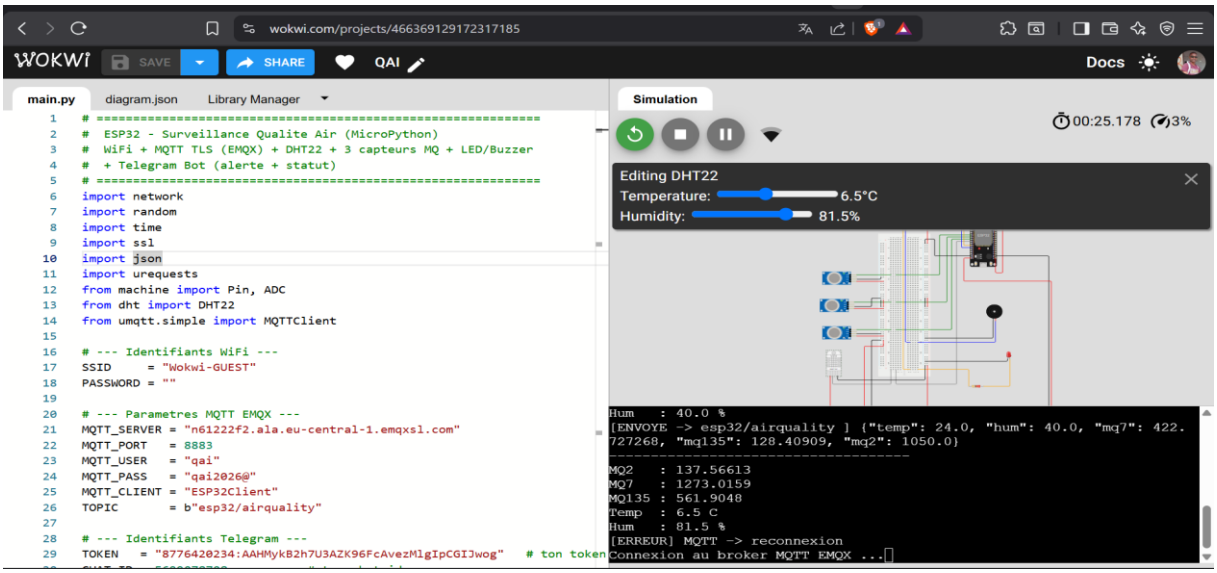


Figure 36 : Interface de Wokwi pendant un troisième test

Après la variation de la température on observe bien sûr un léger changement au niveau des valeurs des capteurs bien qu'ils ne soient pas fixes. Les valeurs observées sont : MQ2=137.56613, MQ7=1273.0159, MQ135=561.9048, Température=6.5 degré et l'humidité 81.5%.

Appliquons aussi une autre forme de variation de la température :

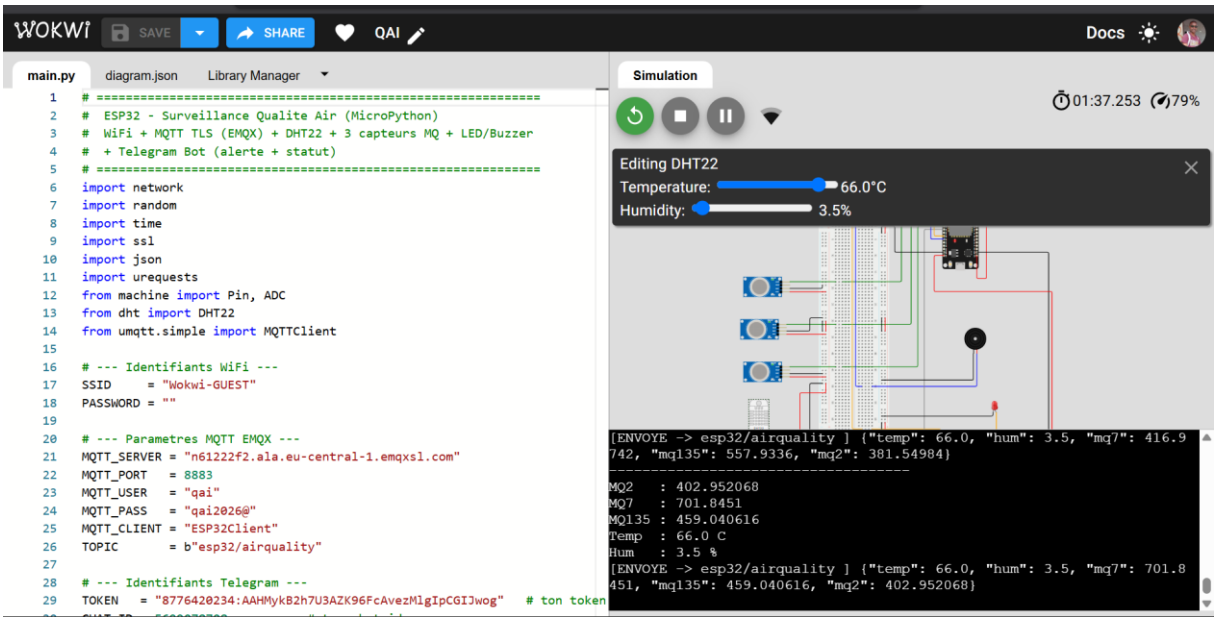


Figure 37 : Interface de Wokwi pendant un quatrième test

Après variation, les données perçues sont également impactées par la température et l'humidité. Avec comme résultat : 402.95 pour le capteur MQ2, 701.84 pour le capteur MQ7 et 459.04 pour le capteur MQ135 sous une température de 66 degré et humidité de 3.5%.

Maintenant vérifions le comportement de notre système en cas de dépassement de seuil :

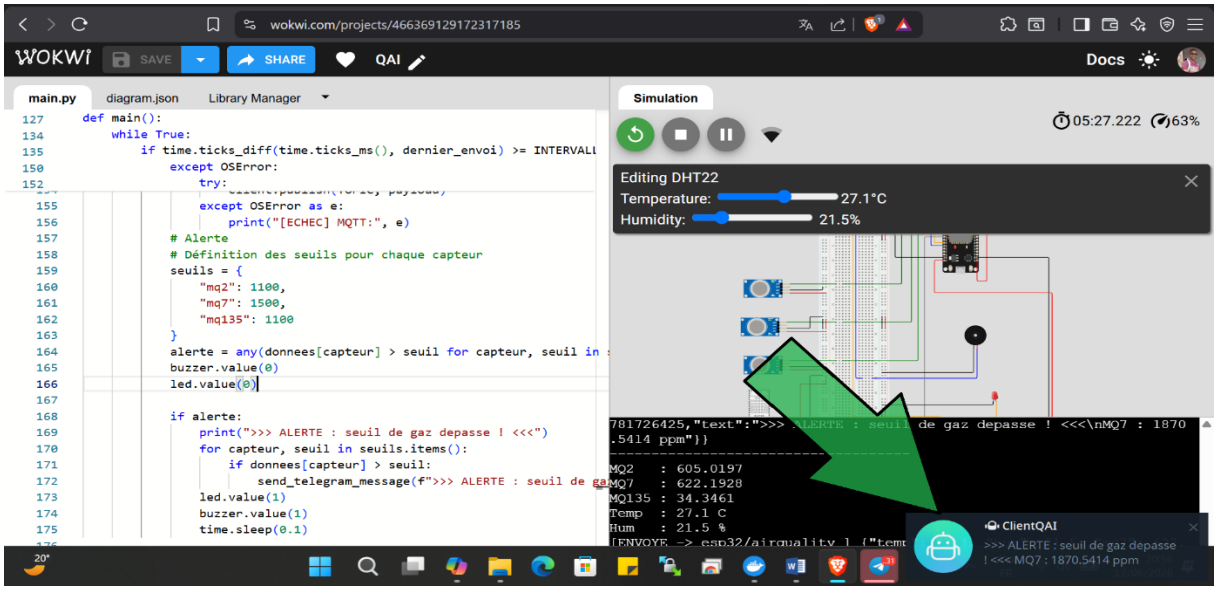


Figure 38: Interface de Wokwi pendant un cinquième test / réception d'une notification Télégram

Une fois qu'un seuil soit dépassé, on reçoit directement une notification télégram via le bot que l'on a créé. La notification renvoie un message accompagné du capteur ayant dépassé le seuil.

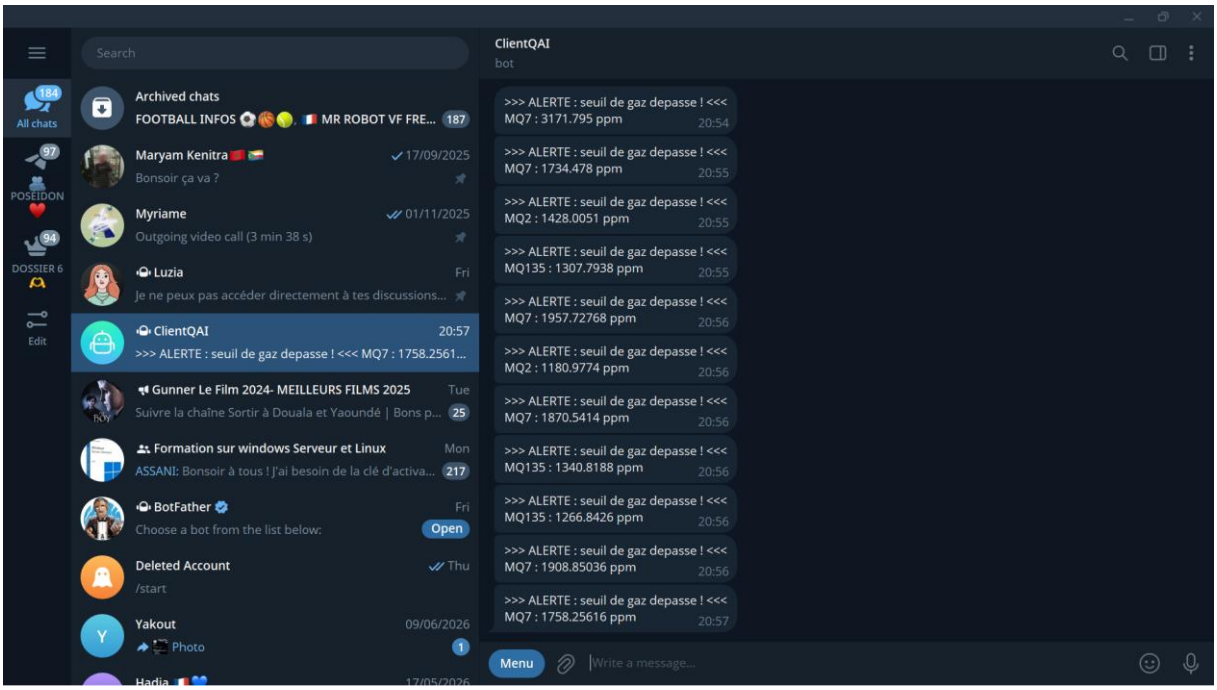


Figure 39 : Interface du Bot Télégram réception des données dépassant le seuil

L'image ci-dessus montre ce qu'on peut voir dans telegram une fois que le seuil soit dépassé.

Pour le suivi en temps réel, nous avons la plateforme AirWatch, un tableau de bord que nous avons créé pour le suivi en temps réel des données de la qualité de l'air intérieur.

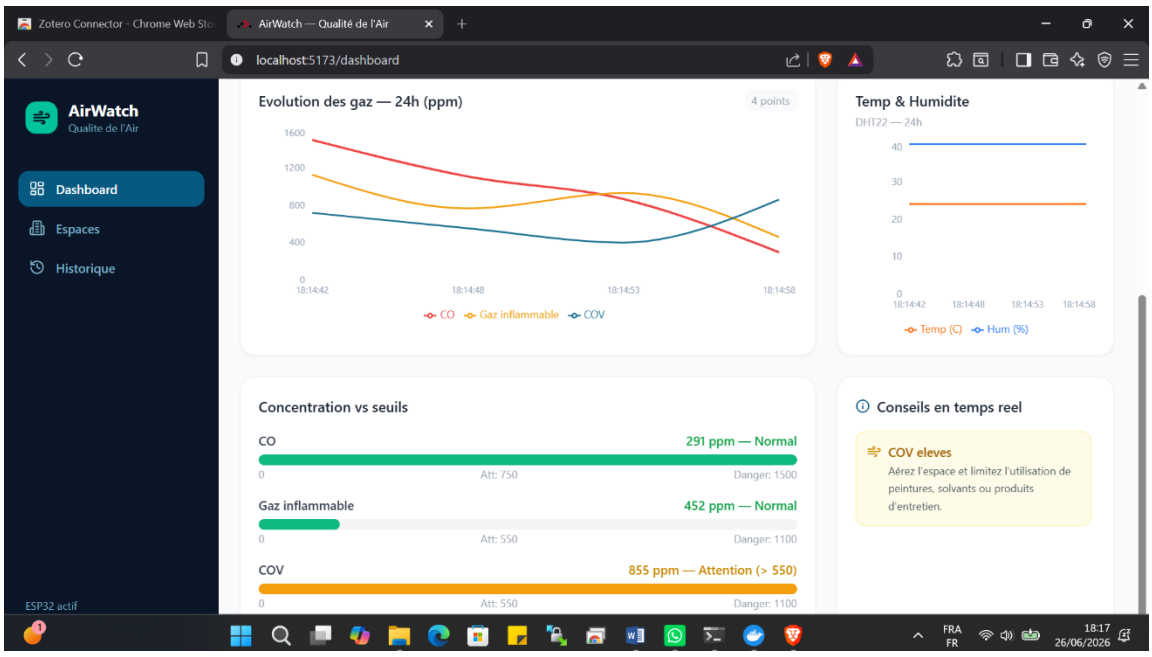


Figure 40 : Interprétation des données dans le tableau de bord

Notre tableau de bord s'auto actualise toute les 5 secondes comme configuré dans notre logiciel embarqué, pour qu'il soit synchronisé avec les données que la base de données reçoit de la carte. Dans l'image, nous observons les différentes mesures prises en compte pour évaluer la qualité de l'air de notre pièce.

3.3 Limites et améliorations

Tout projet comporte inévitablement des limites liées aux ressources, aux contraintes techniques et ou spatio-temporelles. L'analyse des limites n'a pas pour objectif de souligner des faiblesses, mais d'identifier et mettre en évidence des opportunités d'amélioration. En montrant les points à optimiser et en proposant des pistes concrètes, nous assurons une démarche d'évolution continue qui renforce la qualité et la pérennité du projet.

De ce fait, les limites à souligner dans notre projet sont surtout liées aux ressources matérielles, et aux contraintes spatio-temporelles. Les matériaux utilisés bien qu'ils soient précis ou performant ne sont pas d'envergure industrielles que l'on peut trouver dans les systèmes de monitoring de la qualité de l'air extérieur. Ils suscitent surtout un temps de préchauffage allant de 24 à 48h selon le capteur de gaz utilisé, au démarrage du système. Pour le capteur de température et d'humidité, il pet capturer la température jusqu'à une limite allant de -40 à +80 degré Celsius. Dans une région au froid extrême, ce capteur ne peut pas fournir, des données plus que convainquant. Notre système utilise un système d'alerte moins convainquant avec un buzzer et un LED de couleur rouge qui nécessitent que l'utilisateur soit proche de l'appareil pour percevoir l'alerte, et la notification télégram qui dépend de la connexion internet stable et au bon fonctionnement de l'API, une absence de redondance en cas de panne du système d'alerte local et du fait que telegram ne fonctionne que pour un utilisateur ou un petit groupe. Nous avons aussi rencontré des limites à la production du prototype, puisqu'il s'agit de matériaux peu accessible puisqu'il faudrait un certain temps pour retrouver les matériaux et les avoir à disposition à temps pour construire l'appareil.

Pour améliorer les points cités dans le paragraphe précédent, nous pouvons opter pour les solutions suivantes :

- ✓ Pour la contrainte des ressources matérielles :
  - Opter pour des capteurs de gaz industriels
  - Utiliser un capteur de température un peu plus précis
- ✓ Pour le système d'alerte :

- Utiliser plusieurs moyens de notification comme les emails, SMS, ….
- Utiliser plusieurs LED ou différents sons pour différencier les niveaux d'alerte
- Prévoir un second support de stockage de secours en cas d'absence de connexion internet
- Enregistrer chaque alerte dans une base de données pour des futures analyses
- Prévoir la liaison du système de QAI avec le système de ventilation de la salle pour automatisation
- Chiffrer les communications entre la carte et Telegram pour éviter les intrusions
- Le rendre modulable

Nous avons aussi pensé à ajouter un algorithme d'apprentissage automatique qui sera capable à faire des prévisions à court terme, pour anticiper un pic de pollution lié à une vague de chaleur ou trafic intense. Cela peut donner aux occupants une longueur d'avance pour agir et nous avons prévu d'utiliser cette intelligence pour faire de lui un conseiller des utilisateurs pour les accompagner à mieux garder leurs endroits qu'ils ont l'habitude de les fréquenter.

Conclusion du chapitre 3

Les tests réalisés nous ont permis de vérifier la conformité des principales fonctionnalités attendues, notamment le collecte des données par les capteurs, la correction via la température et l'humidité, l'affichage et suivi en temps réel de la qualité de l'air ainsi que le système d'alerte locale et distante. Parmi les limites de notre système, nous avons la non réalisation physique pour manque de ressources toutefois le système a été simulé dans des conditions proche de la réalité et quelques limites liés aux matériaux. Ces observations constituent une base solide pour des améliorations futures pour obtenir non un prototype, mais un appareil répondant aux exigences du marché international.

Gestion de projet

La réalisation de ce projet a suivi une planification structurée en plusieurs phases, s'étalant de février à juillet. Le diagramme ci-dessous présente l'enchaînement des grandes étapes du projet, depuis l'étude bibliographique et le choix des composants jusqu'à la fiabilisation et l'optimisation finale du prototype, en passant par la conception matérielle, le développement logiciel et les tests de validation.

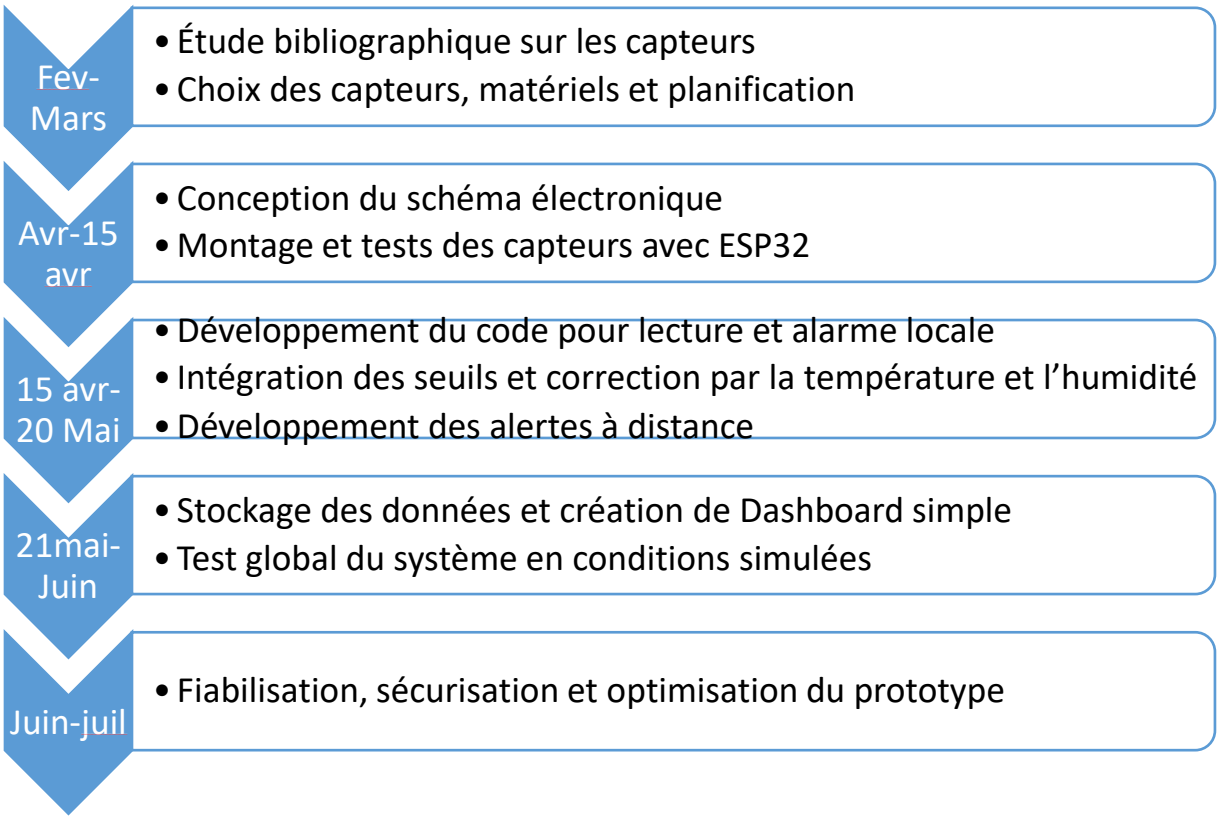


Figure 41 : Diagramme de gestion de projet

Conclusion générale

Ce projet a constitué une opportunité privilégiée de mettre en pratique les compétences acquises en systèmes d'information et transformation digitale. Il a permis d'enclencher l'ensemble du cycle de vie d'un projet d'Internet des Objets (IoT), en partant de l'analyse rigoureuse des besoins jusqu'à la validation expérimentale, tout en passant par la conception matérielle et logicielle, l'intégration embarquée et le développement d'API.

Le prototype ainsi développé démontre la faisabilité technique et la viabilité économique d'une solution complète de surveillance de la qualité de l'air intérieur. Ce système intègre avec succès la détection multi-gaz, la correction algorithmique des mesures, le déclenchement d'alertes locales et distantes, ainsi que le stockage et la visualisation des flux de données en temps réel.

Bien que le dispositif présente certaines limites, notamment liées à la sensibilité intrinsèque des capteurs et à l'absence d'une interface utilisateur avancée, nous pensons que notre travail pose des jalons solides et ouvre des perspectives d'évolution prometteuses. Parmi celles-ci figurent l'intégration native au sein de plateformes cloud, le développement d'applications mobiles dédiées, ou encore l'optimisation des modèles de correction des données.

En définitive, ce projet illustre de manière concrète l'apport de l'IoT et de la transformation digitale dans la résolution de problématiques sociétales majeures, au premier rang desquelles figurent la préservation de la santé et la sécurité des occupants au sein des espaces clos.

Références

[1] « Air quality in Europe 2021 ». Consulté le: 26 juin 2026. [En ligne]. Disponible sur: https://www.eea.europa.eu/en/analysis/publications/air-quality-in-europe-2021

[2] E. P. Agency, « Air quality in Ireland 2023 ». Consulté le: 26 juin 2026. [En ligne]. Disponible sur: https://www.epa.ie/publications/monitoring--assessment/air/air-quality-in-ireland-2023.php?utm_source=copilot.com

[3] J. G. Allen, P. MacNaughton, U. Satish, S. Santanam, J. Vallarino, et J. D. Spengler, « Associations of Cognitive Function Scores with Carbon Dioxide, Ventilation, and Volatile Organic Compound Exposures in Office Workers: A Controlled Exposure Study of Green and Conventional Office Environments », *Environ. Health Perspect.*, vol. 124, nº 6, p. 805-812, juin 2016, doi: 10.1289/ehp.1510037.

[4] « Droit national en vigueur - Codes - Légifrance ». Consulté le: 26 juin 2026. [En ligne]. Disponible sur: https://www.legifrance.gouv.fr/download/pdf?id=RimJ6KYSmxEl-vKBu8qtwJMD0LeooqkyAbZzfe5bjzs=

[5] « Capteurs semi-conducteurs (oxyde métallique): un guide complet - Capteur un arrêt - le fournisseur de capteurs de la meilleure valeur et la chaîne d'approvisionnement ». Consulté le: 26 juin 2026. [En ligne]. Disponible sur: https://sensor1stop.com/fr/knowledge/semiconductor-sensors/

[6] « Lignes directrices mondiales de l'OMS sur la qualité de l'air ». Consulté le: 26 juin 2026. [En ligne]. Disponible sur: https://www.who.int/fr/news-room/questions-and-answers/item/who-global-air-quality-guidelines?utm_source=copilot.com

[7] « Pollution de l'air extérieur. Monographies du CIRC sur l'évaluation des risques cancérogènes pour l'homme, volume 109 - 2013 | Coalition pour le climat et l'air pur ». Consulté le: 26 juin 2026. [En ligne]. Disponible sur: https://www.ccacoalition.org/fr/resources/outdoor-air-pollution-iarc-monographs-evaluation-carcinogenic-risks-humans-volume-109-2013

[8] « ESP32 Datasheet | Espressif Documentation ». Consulté le: 26 juin 2026. [En ligne]. Disponible sur: https://documentation.espressif.com/esp32_datasheet_en.html?utm_source=copilot.com

[9] « Capteurs de gaz: les différents types et utilisation du capteur MQ-136 de sulfure d'hydrogène avec Arduino maroc ». Consulté le: 26 juin 2026. [En ligne]. Disponible sur: https://www.moussasoft.com/utiliser-capteurs-de-gaz-mq-136-2-arduino-maroc/

[10] « DHT22 Module Capteur de Température et Humidité », MicroPlanet Maroc. Consulté le: 26 juin 2026. [En ligne]. Disponible sur: https://www.micro-planet.ma/produit/module-capteur-de-temperature-humidite-dht22/

[11] « Arduino - Buzzer Library | Arduino Tutorial », Arduino Getting Started. Consulté le: 26 juin 2026. [En ligne]. Disponible sur: https://arduinogetstarted.com/tutorials/arduino-buzzer-library

[12] « Wong et al., 2018 ».

Annexes

Cette section regroupe l'ensemble des fiches techniques, descriptions et caractéristiques détaillées des composants matériels et logiciels mis en œuvre dans le cadre du projet de surveillance de la qualité de l'air. Chaque annexe présente, pour le dispositif concerné, son principe de fonctionnement, ses spécifications techniques, ses avantages et ses limites, ainsi que sa justification dans l'architecture globale du système.

Annexe A : Les capteurs

Les capteurs constituent l'interface entre le monde physique et le système électronique. Ils convertissent des grandeurs physico-chimiques (concentration de gaz, température, humidité) en signaux électriques exploitables par le microcontrôleur. Le projet repose sur quatre capteurs complémentaires : trois capteurs de gaz de la famille MQ et un capteur climatique DHT22.

A.1 — Capteur de gaz MQ-2

Le MQ-2 est un capteur de gaz à semi-conducteur très répandu, utilisé pour la détection de gaz combustibles et la surveillance de la qualité de l'air. Son élément sensible est une couche d'oxyde d'étain (SnO_2) portée à haute température par une résistance chauffante interne. En présence d'un gaz réducteur, la résistance de cette couche diminue, ce qui modifie la tension de sortie mesurée. Peu coûteux, robuste et facile à interfacer, il est particulièrement adapté à la détection précoce de fuites et de fumées.

Principe de fonctionnement : la variation de résistance de la couche sensible est convertie en signal analogique via un pont diviseur de tension, puis numérisée par le convertisseur analogique-numérique (CAN) du microcontrôleur.

Tableau 5 : Tableau caractéristique du capteur MQ-2

Caractéristique	Valeur / Description
Type	Capteur de gaz à semi-conducteur (oxyde d'étain SnO_2)
Particularité	Détecteur de gaz inflammables et de fumée
Gaz détectés	GPL, propane, butane, méthane, hydrogène, alcool, fumée
Plage de détection	200 à 10 000 ppm
Sensibilité	Haute sensibilité pour les gaz combustibles
Temps de réponse	Inférieur à 10 s
Tension d'alimentation	5 V DC
Type de sortie	Analogique (AO) et numérique à seuil (DO)
Temps de préchauffage	Environ 20 s (24 h pour une stabilité optimale)
Durée de vie	Supérieure à 5 ans

Avantages : faible coût, grande sensibilité, réponse rapide et large gamme de gaz couverte. Limites : faible sélectivité (répond à plusieurs gaz à la fois), sensibilité à la température et à l'humidité, et nécessité d'un étalonnage préalable pour des mesures quantitatives fiables.

A.2 — Capteur de monoxyde de carbone MQ-7

Le MQ-7 est spécialisé dans la détection du monoxyde de carbone (CO), un gaz inodore, incolore et particulièrement dangereux. Comme le MQ-2, il repose sur une couche d'oxyde d'étain, mais fonctionne selon un cycle thermique alternant une phase de chauffe à haute température (5 V) et une phase de mesure à basse température (1,4 V), ce qui améliore nettement sa sélectivité vis-à-vis du CO.

Tableau 6 : Tableau caractéristique du capteur MQ-7

Caractéristique	Valeur / Description
Type	Capteur de gaz à semi-conducteur (SnO_2) à cycle thermique
Particularité	Détecteur de monoxyde de carbone (CO)
Plage de détection	20 à 2 000 ppm
Sensibilité	Très sensible au monoxyde de carbone
Temps de réponse	Inférieur à 10 s
Tension d'alimentation	5 V DC (chauffage cyclique 5 V / 1,4 V)
Type de sortie	Analogique (AO) et numérique à seuil (DO)
Cycle de mesure	60 s chauffe haute + 90 s mesure basse
Conditions optimales	20 °C, 65 % HR, charge RL ≈ 10 kΩ

Intérêt pour le projet : la détection du CO est essentielle pour la sécurité des personnes dans les espaces clos. Le MQ-7 complète le MQ-2 en ciblant spécifiquement un gaz toxique que ce dernier détecte mal.

A.3 — Capteur de qualité de l'air MQ-135

Le MQ-135 est dédié à la surveillance de la qualité de l'air intérieur. Il réagit à un large éventail de polluants et de composés organiques volatils (COV), ce qui en fait le capteur central pour l'évaluation globale de la pollution ambiante. Il est fréquemment utilisé pour estimer un indice de qualité de l'air ou une concentration équivalente en CO_2.

Tableau 7 : Tableau caractéristique du capteur MQ-135

Caractéristique	Valeur / Description
Type	Capteur de gaz à semi-conducteur (SnO_2)
Particularité	Mesure de la pollution de l'air intérieur (COV) et gaz toxiques
Gaz détectés	NH_3, NOx, alcool, benzène, fumée, CO_2, COV
Plage de détection	10 à 1 000 ppm
Sensibilité	Sensible aux composés organiques volatils (qualité de l'air)
Temps de réponse	Inférieur à 10 s
Tension d'alimentation	5 V DC
Type de sortie	Analogique (AO) et numérique à seuil (DO)
Application typique	Stations de qualité de l'air, ventilation intelligente

Complémentarité : associé au MQ-2 et au MQ-7, le MQ-135 permet de couvrir à la fois les gaz inflammables, le monoxyde de carbone et la pollution générale, offrant ainsi une vision complète de l'environnement surveillé.

A.4 — Capteur de température et d'humidité DHT22

Le DHT22 (également appelé AM2302) mesure la température et l'humidité relative de l'air. Il intègre une thermistance et un capteur d'humidité capacitif reliés à un microcontrôleur interne qui délivre directement une valeur numérique calibrée sur une seule broche de données. Précis, économe en énergie et simple à interfacer, il convient parfaitement aux applications de surveillance environnementale comme ce projet.

Son intérêt est double : il fournit des données climatiques utiles en soi et permet de compenser l'influence de la température et de l'humidité sur les mesures des capteurs MQ, améliorant ainsi la fiabilité globale du système.

Tableau 8 : Tableau caractéristique du capteur de température et d'humidité DHT22

Caractéristique	Valeur / Description
Type	Capteur numérique de température et d'humidité (1 fil)
Plage de température	-40 à +80 °C
Précision (température)	± 0,5 °C
Plage d'humidité	0 % à 100 % d'humidité relative
Précision (humidité)	± 2 % à ± 5 % HR
Résolution	0,1 °C / 0,1 % HR
Fréquence d'échantillonnage	0,5 Hz (1 mesure toutes les 2 s)
Temps de réponse	Inférieur à 2 s
Tension d'alimentation	3,3 à 5,5 V DC
Interface	Numérique à fil unique (One-Wire propriétaire)

A.5 — Tableau comparatif des capteurs

Le tableau suivant synthétise les rôles respectifs des quatre capteurs au sein du système de surveillance.

Tableau 9 : Tableau comparatif des capteurs

Capteur	Grandeur mesurée	Plage	Temps de réponse
MQ-2	Gaz combustibles, fumée	200–10 000 ppm	< 10 s
MQ-7	Monoxyde de carbone (CO)	20–2 000 ppm	< 10 s
MQ-135	Qualité de l'air, COV	10–1 000 ppm	< 10 s
DHT22	Température / humidité	-40 à +80 °C / 0–100 %	< 2 s

Annexe B : Le microcontrôleur

B.1 — Carte ESP32

L'ESP32 est un microcontrôleur moderne développé par Espressif Systems, très utilisé dans les projets de l'Internet des objets (IoT) grâce à sa puissance de calcul et à sa connectivité intégrée. Il combine un processeur double cœur performant, une large gamme d'interfaces d'entrée/sortie et des modules de communication sans fil (Wi-Fi et Bluetooth) au sein d'une seule puce économique. C'est le cœur du système : il collecte les données des capteurs, les traite, puis les transmet vers le serveur via le réseau.

Son architecture intègre également des convertisseurs analogique-numérique (CAN), des interfaces de communication (I^2C, SPI, UART), des broches PWM et des modules de sécurité matérielle, ce qui en fait une plateforme complète pour la conception de dispositifs connectés autonomes.

Tableau 10 : Tableau caractéristique de la carte ESP32

Caractéristique	Valeur / Description
Constructeur	Espressif Systems
Processeur	Xtensa LX6 double cœur, jusqu'à 240 MHz
Mémoire interne	520 Ko SRAM + 448 Ko ROM
Mémoire externe	Flash de 4 Mo à 16 Mo
Wi-Fi	802.11 b/g/n (2,4 GHz), jusqu'à 150 Mbps
Bluetooth	v4.2 BR/EDR + BLE (Bluetooth Low Energy)
Convertisseurs	CAN 12 bits (18 canaux), CNA 8 bits (2 canaux)
Interfaces	I²C, SPI, UART, I²S, CAN, PWM
Broches GPIO	Jusqu'à 34 broches programmables
Sécurité	Accélération matérielle AES, SHA, RSA, ECC
Température de fonctionnement	-40 à +125 °C
Tension d'alimentation	3,3 V (régulateur embarqué 5 V via USB)

B.2 — Justification du choix

L'ESP32 a été retenu pour ce projet en raison de plusieurs atouts décisifs :

- **Connectivité sans fil native** : le Wi-Fi et le Bluetooth sont intégrés, sans module externe.
- **Puissance de calcul** : double cœur à 240 MHz, largement suffisant pour acquérir et traiter plusieurs capteurs en parallèle.
- **Faible consommation** : modes de veille profonde (deep sleep) permettant une autonomie prolongée sur batterie.
- **Richesse des entrées/sorties** : grand nombre de broches et d'interfaces pour connecter capteurs analogiques et numériques.

- **Coût et écosystème** : composant peu onéreux disposant d'une vaste communauté et compatible avec l'environnement Arduino.

Annexe C : Le système d'alerte local

Le système d'alerte local a pour rôle de signaler immédiatement, sur place, tout dépassement d'un seuil de concentration de gaz. Il repose sur deux composants simples et économiques : un avertisseur sonore (buzzer) et une diode électroluminescente (LED) rouge. Cette double signalisation, sonore et visuelle, permet d'alerter les occupants même en cas de bruit ambiant ou de visibilité réduite.

C.1 — Le buzzer

Un buzzer est un dispositif qui produit un son ou un bourdonnement. Il est utilisé comme organe de signalisation : dans ce projet, il se déclenche lorsqu'un gaz est détecté ou dépasse le seuil admissible. Le buzzer actif retenu génère son propre signal sonore dès qu'il est alimenté, ce qui simplifie sa commande par le microcontrôleur.

Tableau 11 : Tableau caractéristique du Buzzer

Caractéristique	Valeur / Description
Type	Buzzer actif (oscillateur intégré)
Rôle	Alerte sonore en cas de dépassement de seuil
Commande	Broche numérique GPIO27 de l'ESP32
Tension d'alimentation	5 V DC
Niveau sonore	Audible à plusieurs mètres en intérieur
Avantage	Signal immédiat, indépendant de la vue

C.2 — La LED de signalisation

La LED rouge constitue le second canal d'alerte. Elle s'allume simultanément au buzzer pour fournir un repère visuel clair de l'état d'alerte. Elle est protégée par une résistance de limitation de courant afin d'éviter sa détérioration.

Tableau 12 : Tableau caractéristique de la LED

Caractéristique	Valeur / Description
Type	Diode électroluminescente (LED) rouge
Rôle	Alerte visuelle en cas de dépassement de seuil
Commande	Broche numérique GPIO26 de l'ESP32
Tension d'alimentation	3,3 V DC
Protection	Résistance série de 220 Ω
Avantage	Repère visuel simple et peu énergivore

Annexe D : Correction des mesures (température et humidité)

La lecture des fiches techniques des capteurs MQ a révélé que leurs mesures sont fortement influencées par la température et l'humidité de l'environnement. Une pièce chaude ou humide peut conduire le capteur à délivrer une valeur erronée, même lorsque l'air est sain. Selon plusieurs études, l'erreur de mesure peut atteindre 30 à 50 % en l'absence de correction. Il est donc indispensable de corriger les valeurs brutes des capteurs.

D.1 — Méthodes de correction étudiées

- **La correction linéaire simple** : ajustement proportionnel des mesures en fonction de la température et de l'humidité.
- **L'utilisation de tables de correspondance** : association de valeurs mesurées à des valeurs corrigées prédéfinies.
- **Des modèles mathématiques plus complexes** : algorithmes intégrant plusieurs paramètres non linéaires.

Pour ce prototype, la correction linéaire simple a été retenue car elle est facile à programmer et suffisante pour valider le concept.

D.2 — Principe de la correction linéaire

La valeur corrigée est obtenue à partir de la concentration brute mesurée par le capteur, ajustée par des termes proportionnels aux écarts de température et d'humidité par rapport à des conditions de référence. Elle s'exprime sous la forme générale :

$$ppm_corrigé = \frac{ppm_brut}{1 + \mathrm{a} * (\mathrm{T} - 25) + \mathrm{b} * (\mathrm{HR} - 50)}$$

Tableau 13 : Tableau significatif de la formule de correction

Terme	Signification
ppm_brut	Concentration mesurée directement par le capteur, avant correction (« valeur brute d'usine »)
T	Température ambiante (°C) fournie par le DHT22
HR	Humidité relative (%) fournie par le DHT22
a, b	Coefficients de correction propres au capteur ou au protocole utilisé
T_réf, HR_réf	Conditions de référence d'étalonnage du capteur

Annexe E : Schéma de câblage et brochage

Le câblage relie chaque capteur et chaque organe d'alerte aux broches appropriées de la carte ESP32. Les capteurs de gaz utilisent leur sortie analogique (AO), reliée aux entrées du convertisseur analogique-numérique de l'ESP32, tandis que le DHT22 et les organes d'alerte utilisent des broches numériques. Le tableau suivant détaille le plan de branchement complet du prototype.

Tableau 14 : Tableau de cablage

Composant	Broche ESP32	Alimentation
MQ-2 (sortie analogique AO)	GPIO34	5 V
MQ-7 (sortie analogique AO)	GPIO35	5 V
MQ-135 (sortie analogique AO)	GPIO32	5 V
DHT22 (data)	GPIO33	3,3 V
LED rouge (anode)	GPIO26	3,3 V (résistance 220 Ω)
Buzzer actif (signal)	GPIO27	5 V

Remarque : les broches GPIO34 et GPIO35 sont des entrées analogiques uniquement, ce qui convient parfaitement à la lecture des capteurs MQ. L'ensemble du montage a été conçu et vérifié à l'aide du logiciel Fritzing avant la simulation.

Annexe F : La communication (protocole MQTT)

F.1 — Le protocole MQTT

MQTT (Message Queuing Telemetry Transport) est un standard de messagerie léger et efficace conçu pour l'Internet des objets. Il repose sur le modèle publication/abonnement (publish/subscribe), qui découple les émetteurs des récepteurs de données. Conçu pour fonctionner sur des réseaux à faible bande passante et avec des microcontrôleurs contraints comme l'ESP32, il constitue un choix idéal pour ce projet de surveillance de la qualité de l'air.

Contrairement au modèle requête/réponse du protocole HTTP, MQTT permet une diffusion asynchrone et temps réel des mesures : dès qu'un capteur publie une nouvelle valeur, tous les abonnés concernés la reçoivent immédiatement, sans interrogation répétée du serveur.

F.2 — Architecture publish/subscribe

L'architecture MQTT se compose de trois éléments principaux :

- **Clients MQTT** : ce sont les capteurs ou les applications qui publient des messages sur des sujets (topics) ou s'y abonnent. Dans ce projet, l'ESP32 agit comme client publieur des mesures.
- **Broker MQTT** : c'est le serveur central qui reçoit tous les messages publiés et les redistribue aux clients abonnés aux sujets correspondants. Il gère les connexions, les autorisations et la qualité de service.
- **Sujets (topics)** : ce sont les étiquettes hiérarchiques (ex. : capteurs/salle1/CO) qui organisent les messages et permettent un filtrage précis des données.

Le transport repose sur le protocole TCP/IP, avec la possibilité de sécuriser les échanges via TLS (chiffrement des communications) et une authentification par identifiant/mot de passe.

F.3 — Niveaux de qualité de service (QoS)

MQTT propose trois niveaux de qualité de service qui déterminent la garantie de livraison des messages :

Tableau 15 : Tableau de la qualité of Service

Niveau	Garantie	Description
QoS 0	Au plus une fois	Message envoyé sans accusé de réception (« fire and forget »)
QoS 1	Au moins une fois	Livraison garantie, avec d'éventuels doublons
QoS 2	Exactement une fois	Livraison garantie sans doublon (le plus fiable, le plus coûteux)

F.4 — Caractéristiques techniques du protocole

Tableau 16 : Tableau caractéristique du protocole MQTT

Caractéristique	Valeur / Description
Modèle	Publication / abonnement (publish/subscribe)
Protocole de transport	TCP/IP (port 1883, ou 8883 avec TLS)
Sécurité	Chiffrement TLS + authentification utilisateur/mot de passe
Taille minimale d'en-tête	2 octets seulement
Qualité de service	QoS 0, 1 et 2
Avantages	Léger, temps réel, faible consommation de bande passante
Cas d'usage	IoT, télémétrie, capteurs distants, domotique

F.5 — Configuration du broker du projet

La carte ESP32 publie les mesures dans un sujet hébergé sur un serveur central (broker). Seuls les clients abonnés à ce sujet peuvent recevoir les données. Le broker utilisé est hébergé sur le

service EMQX Cloud et les échanges sont sécurisés par TLS/SSL ainsi que par une authentification utilisateur.

Tableau 17 : Tableau caractéristique du Broker MQTT

Caractéristique	Valeur / Description
Service de broker	EMQX Cloud (serverless)
Adresse du serveur	n61222f2.ala.eu-central-1.emqxsl.com
Topic de publication	esp32/airquality
Utilisateur	qai
Authentification	Mot de passe + chiffrement TLS/SSL
Sécurité	Seuls les abonnés au topic accèdent aux données

Grâce à cette configuration, les données publiées sur le broker restent confidentielles : aucun tiers non abonné au topic « esp32/airquality » ne peut y accéder.

Annexe G : Plateforme AirWatch et alertes distantes

G.1 — La plateforme web AirWatch

AirWatch est la plateforme web développée pour visualiser en temps réel les données de qualité de l'air. Les mesures publiées sur le broker MQTT sont stockées dans une base de données MySQL nommée « airquality », puis lues et affichées dans un tableau de bord. La plateforme se compose de trois interfaces principales :

- **Le Dashboard** : affiche l'évolution des données dans le temps avec un rafraîchissement automatique toutes les 5 secondes. On y retrouve les courbes des capteurs et leurs valeurs, qui changent de couleur selon le seuil défini.
- **L'interface Espaces** : regroupe, la ou les salles équipées d'un dispositif de surveillance, plusieurs dispositifs pouvant être reliés à la plateforme.
- **L'interface Historique** : conserve l'historique des données depuis la mise en marche du système, avec possibilité d'exporter ces données au format CSV pour des analyses ultérieures.

G.2 — Les alertes distantes par Telegram

Pour avertir l'utilisateur à distance, le système utilise un bot Telegram. Un bot est créé via l'assistant « BotFather », qui fournit une clé d'accès (TOKEN) à l'API Telegram. Cette clé, associée à l'identifiant du chat, est intégrée au programme embarqué de l'ESP32. Lorsqu'un seuil est dépassé, la carte envoie automatiquement un message indiquant le capteur concerné et sa valeur.

Tableau 18 : Tableau caractéristique du système d'alerte distante

Caractéristique	Valeur / Description
Canal d'alerte distante	Application de messagerie Telegram
Création du bot	Assistant BotFather (commande /newbot)
Authentification API	Clé TOKEN + identifiant de chat (chat ID)
Déclenchement	Envoi automatique en cas de dépassement de seuil
Contenu du message	Nom du capteur et valeur ayant dépassé le seuil
Avantage	Notification à distance, gratuite et instantanée

Annexe H : Écosystème logiciel et outils

La réalisation du prototype a mobilisé un ensemble de logiciels, d'outils et de langages de programmation, depuis la conception des schémas jusqu'au déploiement de la plateforme. Le tableau ci-dessous récapitule les principaux outils et leur usage.

Tableau 19 : Tableau caractéristique de l'écosystème logiciel

Outil / Langage	Catégorie	Usage dans le projet
Canva	Design graphique	Diagrammes de fonctionnement, de séquence, schémas en bloc
Fritzing	Conception électronique	Schéma de câblage des composants sur l'ESP32
Visual Studio Code	Éditeur de code	Développement du logiciel embarqué et de la plateforme
MicroPython	Langage embarqué	Programmation de la carte ESP32
draw.io	Diagrammes	Diagramme de cas d'utilisation
Wokwi	Simulation électronique	Tests du système sans matériel physique
Node-RED	Programmation visuelle	Liaison ESP32 ↔ base de données et MQTT
Docker Desktop	Virtualisation	Déploiement des services dans des conteneurs

H.1 — Frameworks de la plateforme AirWatch

La plateforme se divise en deux parties : le frontend (interface visible par l'utilisateur) et le backend (logique serveur et accès aux données).

Tableau 20 : Tableau caractéristique des technologies utilisées

Partie	Technologies utilisées
Frontend	React + Vite, Tailwind CSS (mise en forme), Recharts (graphiques), Lucide React (icônes), React Router (navigation), Socket.IO Client (temps réel)
Backend	Node.js + Express, MQTT, MySQL2, Socket.IO, Telegram Bot API, dotenv, CORS, Nodemon

Annexe I : Comparatif avec les systèmes existants

Avant de concevoir le prototype, plusieurs systèmes existants ont été analysés afin de situer la valeur ajoutée du projet. Le tableau suivant compare ces solutions.

Tableau 21 : Tableau comparatif avec les systèmes existant

Système	Ce qu'il fait	Ce qui manque
Détecteur CO domestique	Alarme locale seulement	Pas d'alerte à distance
Projet AirPi	Mesures + interface web	Coût élevé
Système ESP8266 + MQ-135	Envoi vers le cloud	Pas d'alarme locale
Notre projet (ESP32 + MQ-2/MQ-7/MQ-135 + DHT22)	Alarmes locales et distantes + correction + stockage	Prototype à valider

La valeur ajoutée du projet réside dans la combinaison, à faible coût, de plusieurs fonctions rarement réunies : détection de plusieurs gaz (CO, gaz inflammables, COV), correction des mesures par la température et l'humidité, double alerte (locale et distante), stockage en base de données et visualisation temps réel.

Annexe J : Jeu de valeurs de test (simulation)

La validation du prototype a été réalisée dans un environnement de simulation (Wokwi). Les capteurs de gaz, non dynamiques dans cet environnement, ont été alimentés par une fonction de valeurs aléatoires, tandis que le DHT22 réagissait dynamiquement aux variations de température et d'humidité appliquées. Le facteur de correction a été activé afin de rapprocher les valeurs des conditions réelles. Le tableau suivant présente les relevés obtenus lors de quatre essais successifs.

Tableau 22 : Tableau de valeur des tests de validation

Essai	MQ-2 (ppm)	MQ-7 (ppm)	MQ-135 (ppm)	Temp. (°C)	HR (%)
Test 1	1050,0	422,7	128,4	24,0	40
Test 2	137,6	1273,0	561,9	6,5	81,5
Test 3	403,0	701,8	459,0	66,0	3,5

Ces essais confirment l'influence de la température et de l'humidité sur les mesures des capteurs de gaz : pour des conditions climatiques différentes, les concentrations relevées varient sensiblement. Lors du dépassement d'un seuil, le système a bien déclenché l'alerte locale (buzzer + LED) et envoyé une notification Telegram indiquant le capteur concerné.

Annexe K : Glossaire des abréviations

Le tableau ci-dessous regroupe les principales abréviations et termes techniques employés dans ce rapport.

Tableau 23 : Tableau des abréviations

Sigle	Signification
IoT	Internet of Things — Internet des objets
MQTT	Message Queuing Telemetry Transport
COV	Composés organiques volatils
CO	Monoxyde de carbone
CO_2	Dioxyde de carbone
ppm	Parties par million (unité de concentration)
HR	Humidité relative
CAN	Convertisseur analogique-numérique
GPIO	General Purpose Input/Output — broche d'E/S générique
TCP/IP	Transmission Control Protocol / Internet Protocol
TLS	Transport Layer Security — protocole de chiffrement
QoS	Quality of Service — qualité de service
SnO_2	Dioxyde d'étain (couche sensible des capteurs MQ)